

AI-RESUME INTELLIGENCE HUB

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENT
FOR THE DEGREE OF
Bachelor of Technology In

Computer Science & Engineering
BY

Shourya Saxena (2210303330)
Divyansh Singh (2350303008)
Saalim Yar Khan (2350303039)
Snehi Rastogi (2210303334)
Shivani Gangwar (2350303022)
Abhinav Katheria (LBCS2023041)



Department of Computer Science and
Engineering
Invertis Institute of Engineering and Technology Invertis
University, Bareilly
May-2026

ABSTRACT

The **AI-Resume Intelligence Hub** is an advanced web-based application designed to automate and enhance the recruitment process using Artificial Intelligence. Traditional resume screening methods are time-consuming and often biased due to manual evaluation. This project addresses these issues by implementing modern AI techniques such as Natural Language Processing (NLP), Large Language Models (LLMs), and semantic similarity algorithms.

The system analyzes resumes, extracts key information, and ranks candidates based on job descriptions using Sentence-Transformers. It also includes innovative features such as AI-powered mock interviews using speech recognition and text-to-speech, and a LaTeX-based resume generator for ATS-friendly resumes.

Built using Python, Streamlit, and Docker, the platform ensures scalability, efficiency, and real-time interaction. This tool aims to improve recruitment accuracy, reduce human bias, and provide valuable feedback to job seekers.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Abstract	iii
	List of Figures	v
	List of Abbreviations	vi
1.	Introduction	1
	1.1 Background and Context	1
	1.2 Problem Statement	2
	1.3 Significance and Impact	2
	1.4 Project Overview and Scope	3
	1.5 Objectives and Expected Outcomes	3
2.	Proposed Objectives	5
3.	Literature Review	14
4.	Methodology	22
	4.1 Data Acquisition and Preprocessing	22
	4.2 Feature Extraction	
	4.3 Semantic Matching	
	4.4 Candidate Ranking	
	4.5 Mock Interview System	
	4.6 Resume Generation	
	4.7 System Architecture	
5.	Technologies Used	24
6.	System Design	36
	6.1 System Architecture	36
	6.2 Module Description	41
7.	Implementation	43
8.	Testing and Results	43
9.	Conclusion	43
	9.1 Future Scope	44
10.	References	

LIST OF ABBREVIATIONS

ABBREVIATIONS	MEANING
Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
LLM	Large Language Model
ATS	Applicant Tracking System
UI	User Interface
UX	User Experience
PDF	Portable Document Format
NER	Named Entity Recognition
POS	Part of Speech
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
CPU	Central Processing Unit
GPU	Graphics Processing Unit
OCR	Optical Character Recognition
TTS	Text-to-Speech
STT	Speech-to-Text
DB	Database
JSON	JavaScript Object Notation
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
SQL	Structured Query Language
KPI	Key Performance Indicator
Saas	Software as a Service

Chapter 1

INTRODUCTION

The AI Resume Intelligence Hub is an innovative platform developed to modernize and streamline the recruitment process using advanced Artificial Intelligence technologies. In today's digital era, organizations receive a vast number of job applications for a single position, making manual resume screening a time-consuming and inefficient task. Traditional Applicant Tracking Systems (ATS) often rely on keyword-based filtering, which fails to capture the true context and meaning of a candidate's qualifications. This project addresses these limitations by incorporating intelligent, context-aware analysis.

The system utilizes Natural Language Processing (NLP), Large Language Models (LLMs), and semantic similarity techniques to evaluate resumes in a more accurate and meaningful way. By comparing resumes with job descriptions using advanced algorithms, it identifies the most suitable candidates based on skills, experience, and relevance rather than just keywords. This ensures a more fair and unbiased selection process.

In addition to resume screening, the platform offers valuable features such as an automated resume builder that generates ATS-friendly documents and an AI-powered mock interview system that provides real-time feedback to users. These features not only assist recruiters but also help candidates improve their performance and readiness.

The AI Resume Intelligence Hub is designed as a scalable and user-friendly web application, integrating modern tools and technologies. Overall, the system aims to reduce manual effort, improve hiring efficiency, and create a transparent recruitment ecosystem that benefits both organizations and job seekers.

1.1 Background and Context

In the modern digital era, the recruitment landscape has undergone a significant transformation due to the rapid growth of online job platforms and global connectivity. Organizations now receive a large volume of job applications for every open position, making the hiring process increasingly complex and time-consuming. Traditional methods of resume screening, which rely heavily on manual evaluation, are no longer efficient in handling such scale. Recruiters often spend a considerable amount of time reviewing resumes, which can lead to delays and inconsistencies in candidate selection.

With the introduction of Applicant Tracking Systems (ATS), the screening process became partially automated; however, these systems primarily depend on keyword matching and lack the ability to understand the contextual meaning of resumes. As a result, qualified candidates may be overlooked if their resumes do not contain specific keywords, while less suitable candidates may be shortlisted based on superficial matches.

Advancements in Artificial Intelligence, particularly in Natural Language Processing (NLP) and Large Language Models (LLMs), have opened new possibilities for intelligent and context-aware resume analysis. These technologies enable systems to interpret unstructured textual data more effectively, understand semantic relationships, and provide deeper insights into candidate profiles.

The AI Resume Intelligence Hub is developed in this context to address the limitations of traditional recruitment systems. It leverages modern AI techniques to enhance accuracy, efficiency, and fairness in the hiring process while also supporting candidates in improving their professional profiles.

1.2 Problem Statement

The modern recruitment process faces significant challenges due to the increasing number of job applications for each position. Recruiters often have to manually review hundreds of resumes, making the process time-consuming, inefficient, and prone to errors. This can lead to delays in hiring and the possibility of overlooking qualified candidates.

Although Applicant Tracking Systems (ATS) are used to automate screening, they mainly rely on keyword-based matching and fail to understand the actual context of a candidate's skills and experience. As a result, suitable candidates may be rejected if their resumes do not match specific keywords, while less qualified candidates may be selected.

Additionally, bias in decision-making and the lack of constructive feedback for applicants further complicate the recruitment process. Therefore, there is a need for an intelligent system that can accurately evaluate resumes, reduce bias, and provide meaningful insights to improve hiring efficiency and fairness.

1.3 Significance and Impact

The AI Resume Intelligence Hub holds significant importance in transforming the traditional recruitment process by introducing efficiency, accuracy, and fairness through advanced technologies. In today's competitive job market, organizations struggle to manage large volumes of applications, often leading to delayed hiring decisions and increased workload for recruiters. This system addresses these challenges by automating resume screening, thereby reducing manual effort and saving valuable time and resources.

One of the key impacts of this project is the improvement in the quality of candidate selection. By using intelligent algorithms such as Natural Language Processing (NLP) and semantic analysis, the system evaluates resumes based on actual skills and experience rather than just keywords. This leads to more accurate shortlisting and ensures that deserving candidates are not overlooked. Additionally, the reduction of human bias in the screening process promotes fairness and transparency in recruitment.

The platform also provides direct benefits to job seekers. Features like automated resume generation and AI-powered mock interviews help candidates improve their profiles and prepare effectively for real-world hiring scenarios. This enhances their confidence and increases their chances of success.

From an organizational perspective, the system supports faster decision-making and improves overall productivity. It can be scaled to handle large datasets and integrated into modern hiring workflows. Overall, the AI Resume Intelligence Hub contributes to creating a more efficient, unbiased, and intelligent recruitment ecosystem that benefits both employers and candidates.

1.4 Project Overview and Scope

The AI Resume Intelligence Hub is a comprehensive web-based platform designed to enhance and automate the recruitment process through the integration of advanced Artificial Intelligence technologies. The system focuses on analyzing resumes, matching them with job descriptions, and providing intelligent insights to both recruiters and candidates. It is developed to overcome the inefficiencies of traditional hiring methods by introducing a more accurate, scalable, and user-friendly solution.

At its core, the platform accepts resumes in digital formats such as PDF and processes them using Natural Language Processing (NLP) techniques. Key information such as skills, education, and work experience is extracted and structured for further analysis. The system then applies semantic similarity algorithms, such as Sentence-Transformers, to compare the content of resumes with job descriptions. This allows the platform to evaluate candidates based on contextual relevance rather than simple keyword matching, resulting in more precise candidate ranking.

In addition to resume screening, the project includes an automated resume builder that generates professional, ATS-friendly resumes using predefined templates. This feature assists candidates in improving the quality and presentation of their resumes. Another important component is the AI-powered mock interview module, which uses speech recognition and text-to-speech technologies to simulate real interview scenarios and provide feedback to users.

The scope of the project extends to both recruiters and job seekers. For recruiters, it offers a streamlined process for shortlisting candidates efficiently, while for candidates, it provides tools for skill enhancement and interview preparation. The system is designed to be scalable and can be deployed across different environments using containerization technologies like Docker.

Overall, the AI Resume Intelligence Hub serves as an integrated solution that bridges the gap between employers and job seekers, improving hiring accuracy, reducing time consumption, and promoting a more transparent and effective recruitment ecosystem.

1.5 Objectives and Expected Outcomes

The primary objective of the AI Resume Intelligence Hub is to develop an intelligent and automated system that enhances the efficiency, accuracy, and fairness of the recruitment

process. The project aims to overcome the limitations of traditional resume screening methods by integrating advanced Artificial Intelligence technologies such as Natural Language Processing (NLP), Large Language Models (LLMs), and semantic similarity algorithms. By doing so, the system seeks to provide a more reliable and context-aware evaluation of candidate profiles.

One of the key objectives is to automate the process of resume analysis by extracting essential information such as skills, education, and work experience from unstructured data. This structured data is then used to compare resumes with job descriptions using semantic matching techniques, ensuring that candidates are evaluated based on relevance rather than just keyword presence. Another important objective is to minimize human bias in recruitment by implementing a standardized and data-driven evaluation approach.

The project also aims to provide additional support to job seekers through features such as an automated resume builder and AI-powered mock interview system. The resume builder helps users create professional, ATS-friendly resumes, while the mock interview module enables candidates to practice real-world interview scenarios and receive constructive feedback. These features are designed to improve candidate readiness and confidence.

From a technical perspective, the system is intended to be scalable, modular, and easy to deploy. The use of modern frameworks and containerization technologies ensures that the platform can handle large volumes of data and operate efficiently across different environments. The user interface is designed to be simple and interactive, providing a seamless experience for both recruiters and candidates.

The expected outcomes of this project include a significant reduction in the time required for resume screening, improved accuracy in candidate selection, and enhanced transparency in the hiring process. Additionally, the system is expected to provide meaningful insights and feedback to candidates, helping them improve their profiles and performance. Overall, the AI Resume Intelligence Hub aims to create a smarter, faster, and more effective recruitment ecosystem that benefits all stakeholders involved.

Chapter 2

Proposed Objectives

The AI Resume Intelligence Hub is conceptualized as a smart, integrated platform that enhances the recruitment process by leveraging modern Artificial Intelligence technologies. The proposed objectives of this project are designed to address the shortcomings of traditional hiring systems while providing a comprehensive solution for both recruiters and job seekers. These objectives focus on automation, accuracy, fairness, scalability, and user engagement, ultimately aiming to create a more efficient and transparent recruitment ecosystem.

2.1 Intelligent Resume Analysis

The primary objective of the system is to develop an intelligent mechanism for analyzing resumes. Traditional resume screening methods rely heavily on manual effort or keyword-based filtering, which often fails to capture the true meaning of a candidate's profile.

The proposed system aims to:

- Extract relevant information such as skills, education, and experience from unstructured resumes
- Interpret the context of the content using Natural Language Processing (NLP)
- Provide a structured representation of candidate data

This ensures that resumes are evaluated more accurately, reducing the chances of overlooking qualified candidates.

2.2 Semantic Matching and Candidate Ranking

A crucial objective of the platform is to implement semantic matching between resumes and job descriptions. Instead of relying on exact keyword matches, the system uses advanced algorithms to understand the meaning behind the text.

Key goals:

- Convert text into vector embeddings using Sentence-Transformers
- Compare resumes with job descriptions using cosine similarity
- Generate a relevance score for ranking candidates

This method allows the system to identify candidates whose skills and experience align closely with job requirements, even if different wording is used. As a result, the accuracy of candidate selection is significantly improved.

2.3 Automation of Recruitment Process

Automation is a central objective of the AI Resume Intelligence Hub. The system is designed to reduce the dependency on manual screening and streamline the entire recruitment workflow.

Automated processes include:

- Resume upload and parsing
- Data extraction and preprocessing
- Candidate evaluation and ranking
- Result generation and visualization

By automating these tasks, the system reduces human workload, minimizes errors, and speeds up the hiring process. This leads to increased efficiency and productivity for organizations.

2.4 Reduction of Bias in Hiring

Bias in recruitment is a major concern that affects fairness and diversity. The proposed system aims to minimize bias by implementing a data-driven evaluation approach.

Objectives:

- Eliminate subjective judgment in initial screening
- Standardize evaluation criteria
- Focus on measurable attributes such as skills and experience

This ensures that all candidates are evaluated on equal grounds, promoting fairness and

transparency in the hiring process.

2.5 AI-Based Resume Builder

Another important objective is to provide a smart resume builder that helps candidates create professional and ATS-friendly resumes.

Features:

- Predefined templates (LaTeX-based)
- Automated formatting
- Real-time preview

This tool assists candidates in presenting their information effectively, increasing their chances of being shortlisted. It also ensures that resumes meet industry standards.

2.6 Interactive Mock Interview System

The platform includes an AI-powered mock interview module designed to simulate real interview scenarios.

Objectives:

- Conduct voice-based interviews using speech recognition
- Generate intelligent interview questions
- Provide real-time feedback

This feature helps candidates improve their communication skills and gain confidence. It also prepares them for real-world interviews by offering a realistic practice environment.

2.7 Real-Time Feedback and Suggestions

Providing meaningful feedback is a key objective of the system. Unlike traditional recruitment processes, this platform offers insights to candidates.

Objectives:

- Identify skill gaps
- Suggest improvements in resumes
- Provide performance analysis

This helps candidates understand their strengths and weaknesses, enabling them to improve their profiles and enhance their employability.

2.8 Scalable and Modular System Design

The system is designed to be scalable and adaptable to different use cases.

Objectives:

- Use modular architecture
- Implement containerization using Docker
- Ensure compatibility across platforms

This allows the system to handle large volumes of data and be deployed in various environments without performance issues.

2.9 User-Friendly Interface

A well-designed user interface is essential for the success of the platform. The system aims to provide a smooth and interactive user experience.

Goals:

- Simple and intuitive navigation
- Clean and modern design
- Interactive dashboards

This ensures that both recruiters and candidates can use the platform بسهولة (easily) without technical complexity.

2.10 Data Visualization and Insights

The system aims to present data in a visually understandable format.

Objectives:

- Display candidate rankings through charts
- Provide skill distribution graphs
- Generate performance dashboards

These visual insights help recruiters make informed decisions quickly and efficiently.

2.11 Integration of Advanced AI Technologies

The platform integrates modern AI tools to enhance its functionality.

Objectives:

- Use Large Language Models (LLMs) for intelligent evaluation
- Implement LangChain for workflow orchestration
- Apply NLP techniques for text analysis

This ensures that the system remains future-ready and capable of handling complex tasks.

2.12 End-to-End Recruitment Solution

The ultimate objective is to develop a complete recruitment and career assistance platform.

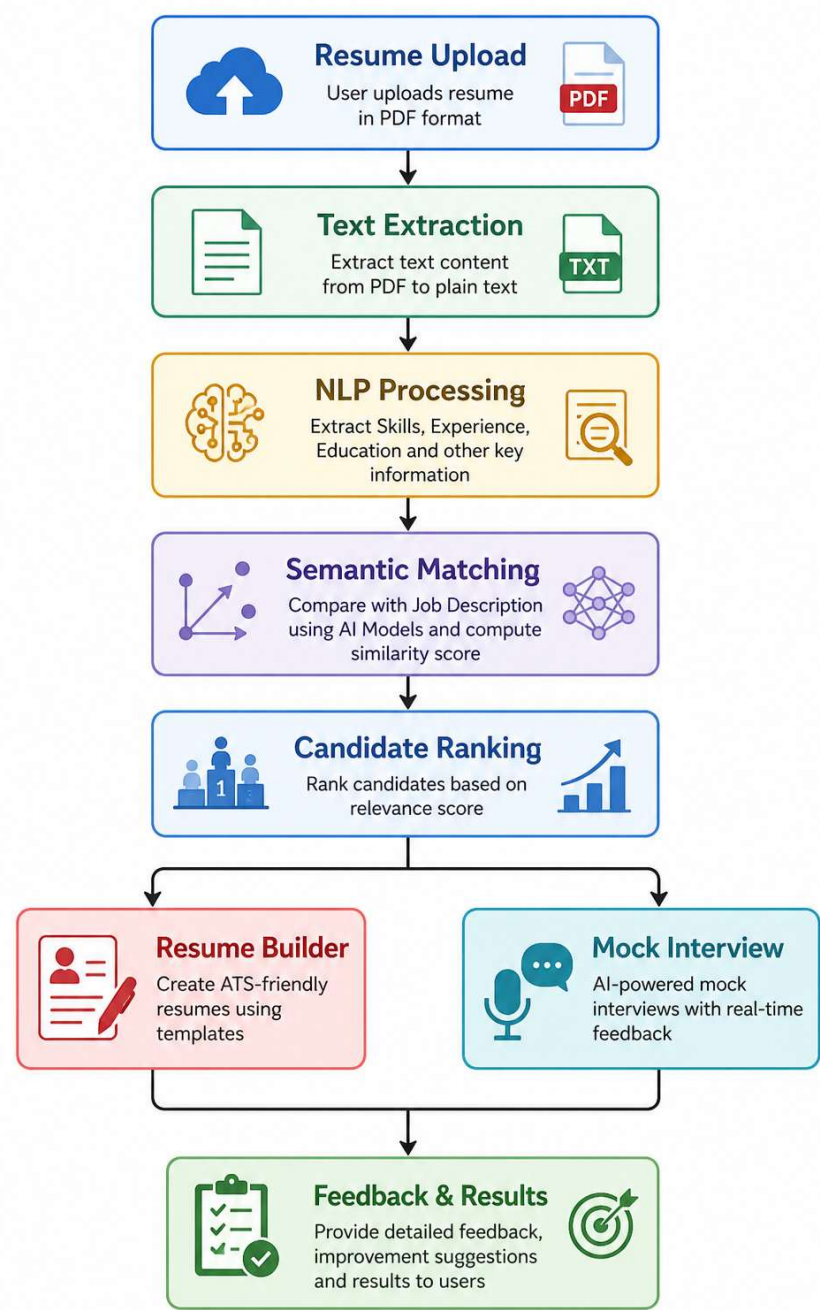
Features:

- Resume screening
- Resume building
- Mock interviews
- Feedback system

This creates a unified environment where both recruiters and candidates can benefit from advanced tools and insights.

2.13 System Workflow Diagram

Below is a simple diagram illustrating the working of the system:



Chapter 3

Literature

Review

The rapid advancement of Artificial Intelligence (AI) and its integration into various domains has significantly transformed traditional processes, including recruitment. The literature on intelligent recruitment systems highlights the evolution from manual resume screening to automated and AI-driven approaches. This section reviews existing research, technologies, and methodologies relevant to the development of the AI Resume Intelligence Hub, focusing on resume parsing, Natural Language Processing (NLP), semantic matching, and AI-based candidate evaluation.

3.1 Traditional Recruitment Systems

In the early stages of recruitment, organizations relied entirely on manual screening of resumes. Recruiters would review each application individually, which was not only time-consuming but also prone to human error and bias. As the number of job applications increased due to digital job portals, manual screening became inefficient and unsustainable.

To address these challenges, Applicant Tracking Systems (ATS) were introduced. ATS systems automated parts of the recruitment process by filtering resumes based on predefined keywords. However, research indicates that these systems have significant limitations. They lack the ability to understand context and semantics, leading to inaccurate filtering. For example, a candidate with relevant skills may be rejected if their resume does not contain specific keywords.

Thus, while ATS systems improved efficiency to some extent, they failed to provide accurate and meaningful evaluation of candidate profiles.

3.2 Natural Language Processing in Recruitment

Natural Language Processing (NLP) has emerged as a powerful tool for analyzing

unstructured text data, such as resumes and job descriptions. NLP techniques enable systems to extract meaningful information, including skills, education, experience, and other relevant attributes.

Research studies have demonstrated the effectiveness of NLP in resume parsing and information extraction. Tools such as Named Entity Recognition (NER) and Part-of-Speech (POS) tagging are commonly used to identify key entities and relationships within text. These techniques allow for the conversion of unstructured resumes into structured data that can be analyzed efficiently.

Furthermore, NLP enables systems to understand the context of words rather than relying solely on exact matches. This capability is crucial for improving the accuracy of resume screening systems. The integration of NLP in recruitment systems has been shown to significantly reduce manual effort and enhance decision-making.

3.3 Semantic Similarity and Machine Learning Models

One of the major advancements in AI-based recruitment systems is the use of semantic similarity techniques. Traditional keyword-based systems fail to capture the meaning behind text, whereas semantic models analyze the relationship between words and phrases.

Machine Learning models, particularly transformer-based architectures, have revolutionized text analysis. Models such as BERT (Bidirectional Encoder Representations from Transformers) and Sentence-Transformers are widely used for generating text embeddings. These embeddings represent the semantic meaning of text in numerical form, allowing for accurate comparison between resumes and job descriptions.

Research shows that cosine similarity is an effective method for measuring the similarity between text embeddings. By applying this technique, systems can rank candidates based on how closely their profiles match job requirements. This approach significantly improves the accuracy of candidate selection compared to traditional methods.

3.4 Large Language Models (LLMs) in Recruitment

Large Language Models (LLMs) have further enhanced the capabilities of AI-driven recruitment systems. These models, trained on vast amounts of data, can understand complex language patterns and generate human-like responses.

Recent studies highlight the use of LLMs in resume evaluation, candidate assessment, and feedback generation. LLMs can analyze resumes in detail, identify strengths and weaknesses, and provide personalized recommendations. This capability is particularly useful for improving candidate experience.

Additionally, LLMs can assist recruiters by summarizing candidate profiles, generating interview questions, and providing insights into candidate suitability. The integration of LLMs in recruitment systems has been shown to improve both efficiency and accuracy.

3.5 Resume Parsing Technologies

Resume parsing is a critical component of modern recruitment systems. It involves extracting structured information from resumes, such as contact details, skills, education, and work experience.

Various techniques have been proposed for resume parsing, including rule-based approaches, machine learning models, and hybrid methods. Rule-based systems rely on predefined patterns, which may not be flexible enough to handle diverse resume formats. Machine learning-based approaches, on the other hand, can adapt to different formats and improve accuracy over time.

Research indicates that combining NLP techniques with machine learning models yields the best results in resume parsing. Tools such as spaCy and NLTK are commonly used for implementing these techniques. The development of robust resume parsing systems is essential for enabling accurate data extraction and analysis.

3.6 AI-Based Candidate Ranking Systems

Candidate ranking is a crucial step in the recruitment process. Traditional systems often rank candidates based on keyword frequency, which may not reflect their actual suitability.

AI-based ranking systems use advanced algorithms to evaluate candidates based on multiple factors, including skills, experience, and relevance to job descriptions. Research suggests that combining semantic similarity with machine learning models leads to more accurate ranking results.

These systems can also incorporate additional features such as experience weighting and skill importance to refine the ranking process. By providing a more comprehensive evaluation, AI-based ranking systems help recruiters make better decisions.

3.7 Bias and Fairness in AI Recruitment

Bias in recruitment is a widely discussed issue in the literature. Traditional hiring processes are often influenced by unconscious biases, which can lead to unfair outcomes.

AI-based systems have the potential to reduce bias by standardizing evaluation criteria and focusing on objective data. However, research also highlights the risk of bias in AI models if they are trained on biased data.

To address this issue, researchers emphasize the importance of ethical AI practices, including:

- Using diverse and unbiased datasets
- Regularly auditing models
- Ensuring transparency in decision-making

The AI Resume Intelligence Hub incorporates these principles to promote fairness and inclusivity in recruitment.

3.8 AI in Interview Preparation

Recent advancements in AI have enabled the development of systems for interview preparation. AI-powered mock interview platforms use speech recognition and natural language understanding to simulate real interview scenarios.

These systems can analyze candidate responses, provide feedback, and suggest improvements. Research shows that such platforms significantly enhance candidate

confidence and performance.

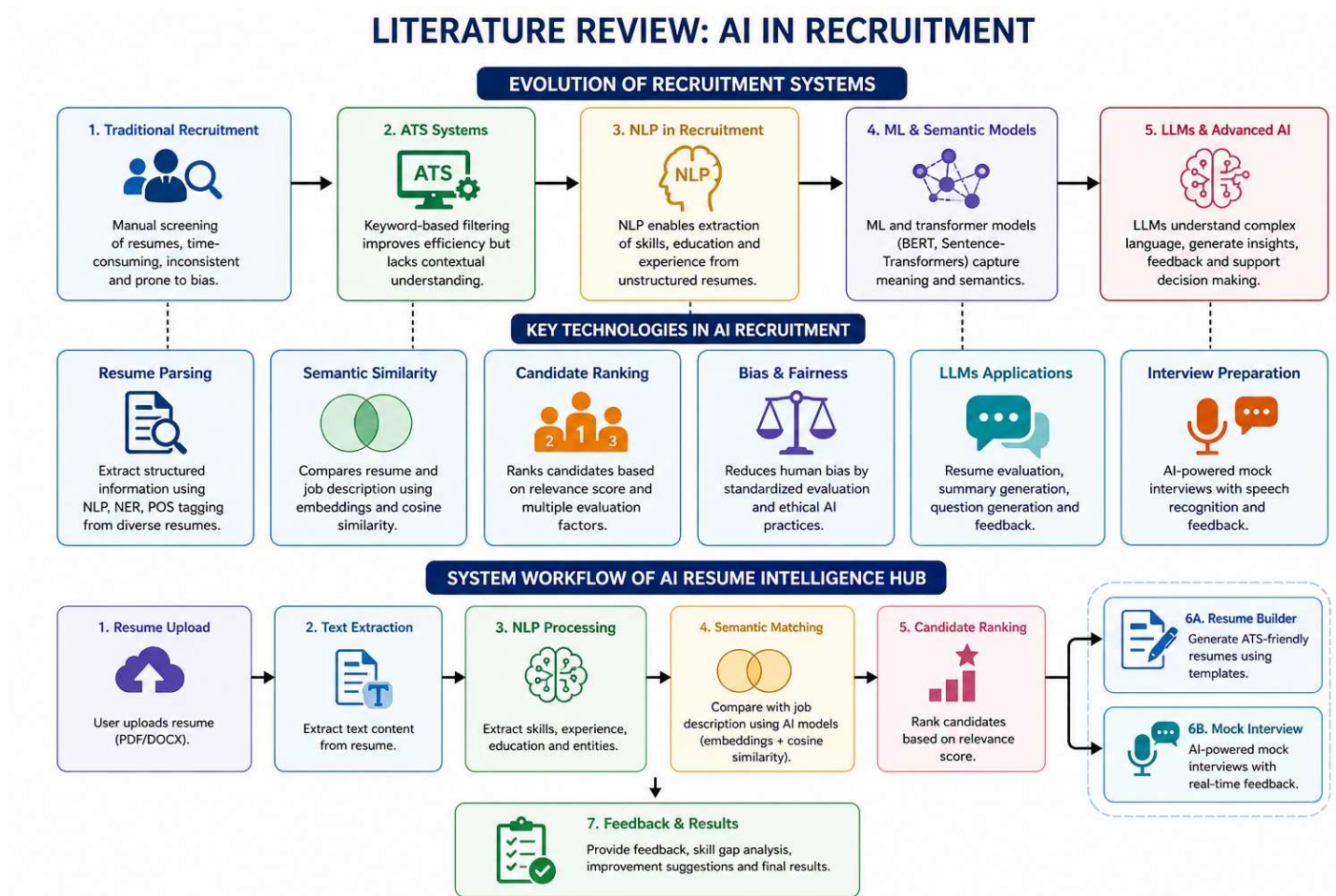
The integration of AI-based interview preparation tools in recruitment platforms adds value by supporting candidates throughout their job search journey.

3.9 Limitations of Existing Systems

Despite significant advancements, existing recruitment systems still face several challenges:

- Lack of contextual understanding in some systems
- Limited adaptability to diverse resume formats
- Potential bias in AI models
- High computational requirements

These limitations highlight the need for more advanced and integrated solutions.



Chapter 4

Methodology

The methodology of the AI Resume Intelligence Hub defines a structured process for analyzing resumes and generating meaningful insights for recruitment. It begins with data acquisition, where resumes and job descriptions are collected in digital formats such as PDF. Since these documents contain unstructured data, text extraction techniques are applied to convert them into readable text. The extracted data is then preprocessed by removing noise, stop words, and unnecessary symbols while applying normalization methods like tokenization and lemmatization.

Following preprocessing, feature extraction is performed using Natural Language Processing techniques. Important details such as skills, education, and work experience are identified and converted into a structured format. This structured data enables efficient comparison between candidates and job requirements.

The system then performs semantic matching, where both resumes and job descriptions are transformed into vector representations using advanced models. Similarity between them is calculated to determine how closely a candidate's profile matches the job role. Based on these similarity scores, candidates are ranked accordingly, helping recruiters identify the most suitable applicants quickly.

Additionally, the methodology incorporates a mock interview system that allows candidates to practice interviews and receive AI-based feedback. A resume generation module is also included to help users create professional, structured resumes.

Overall, the methodology integrates data processing, AI techniques, and user-oriented features to deliver an efficient and intelligent recruitment solution.

4.1 Data Acquisition and Preprocessing

The first step in the methodology involves collecting resumes and job descriptions in digital formats, primarily PDF. These documents contain unstructured textual data, which must be converted into a usable format. Text extraction techniques are applied to convert PDF content into plain text.

Once extracted, the data undergoes preprocessing to improve quality and consistency. This includes removing unnecessary symbols, formatting issues, and irrelevant information. Tokenization is performed to break text into smaller units, while stop words are removed to

retain only meaningful words. Normalization techniques such as lowercasing and lemmatization are also applied. This step ensures that the data is clean, structured, and ready for further analysis.

4.2 Feature Extraction

After preprocessing, the system extracts key features from the resume text. These features include essential information such as skills, educational qualifications, work experience, and certifications.

Natural Language Processing techniques like Named Entity Recognition (NER) are used to identify and categorize relevant entities. The extracted features are then structured into a standardized format, making it easier for the system to analyze and compare candidate profiles. This step plays a crucial role in transforming raw text into meaningful data.

4.3 Semantic Matching

Semantic matching is a core component of the methodology. In this step, the system compares resumes with job descriptions to determine their relevance. Instead of relying on keyword matching, advanced models such as Sentence-Transformers are used to generate vector representations of text.

These vectors capture the contextual meaning of the content. Cosine similarity is then applied to measure the similarity between resume and job description embeddings. This approach allows the system to identify candidates whose profiles align closely with job requirements, even if different terminology is used.

4.4 Candidate Ranking

Based on the similarity scores obtained from semantic matching, candidates are ranked in order of relevance. The system assigns a score to each resume, reflecting how well it matches the job description.

Additional factors such as experience level and skill importance can also be considered to refine the ranking process. This step ensures that recruiters receive a prioritized list of candidates, making the selection process faster and more efficient.

4.5 Mock Interview System

The methodology includes an AI-powered mock interview module designed to help candidates prepare for real interviews. This system uses speech recognition to capture user responses and converts them into text.

Artificial Intelligence models analyze these responses and generate feedback based on content,

clarity, and relevance. Text-to-speech technology is used to simulate interview questions, creating an interactive experience. This module enhances candidate confidence and communication skills.

4.6 Resume Generation

The system also provides a resume generation feature to assist candidates in creating professional and ATS-friendly resumes. Users can input their details, which are then structured into predefined templates.

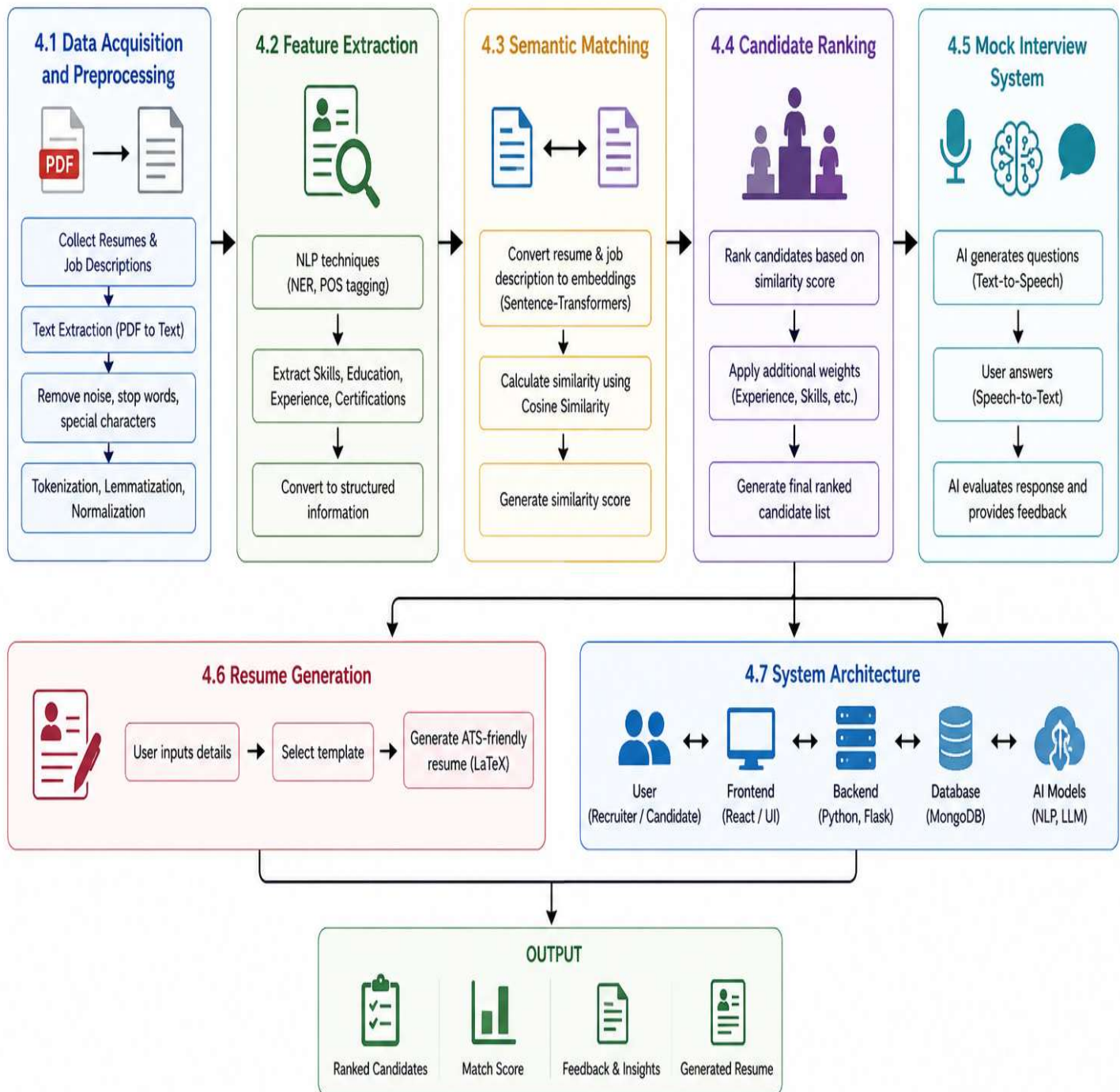
LaTeX-based formatting is used to ensure high-quality and standardized output. This feature helps candidates improve the presentation of their resumes, increasing their chances of being shortlisted.

4.7 System Architecture

The overall system architecture is designed to be modular and scalable. The frontend is developed using user-friendly interfaces, allowing easy interaction for both recruiters and candidates. The backend handles data processing, AI model integration, and result generation.

Different modules such as resume parsing, semantic matching, ranking, and interview systems work together in a coordinated manner. The system can be deployed using containerization technologies, ensuring flexibility and efficient performance across different environments.

4. METHODOLOGY



Chapter 5

Technologies Used

Below is a comprehensive explanation of all technologies used in the **AI Resume Intelligence Hub**, presented in detailed points with theoretical depth.

- **Python (Core Programming Language)**

Python serves as the backbone of the entire system, enabling the development of all major functionalities. It is widely preferred in Artificial Intelligence and data-driven applications due to its simplicity, readability, and vast ecosystem of libraries. Python supports rapid development and prototyping, allowing developers to build and test models efficiently. Its compatibility with AI frameworks and ease of integration with web technologies make it ideal for building a full-stack intelligent system. Additionally, Python handles backend logic, data processing, model execution, and system orchestration seamlessly, ensuring that different modules communicate effectively within the application.

- **Natural Language Processing (NLP)**

Natural Language Processing is a critical technology that enables the system to understand and interpret human language. Since resumes and job descriptions are written in unstructured textual form, NLP techniques are used to convert them into structured and meaningful data. NLP allows the system to identify patterns, relationships, and context within text. It plays a vital role in extracting key information such as skills, qualifications, and experience. By understanding the semantics of language, NLP improves the accuracy of resume analysis and ensures that candidates are evaluated based on meaningful insights rather than surface-level information.

- **spaCy (Advanced NLP Library)**

spaCy is used for high-performance NLP processing. It provides advanced capabilities such as Named Entity Recognition, dependency parsing, and part-of-speech tagging. These features help in identifying important elements within resumes, such as organization names, dates, roles, and technical skills. spaCy is optimized for speed and efficiency, making it suitable for processing large volumes of resumes. Its pre-trained models and customization options allow the system to adapt to different types of data and improve accuracy over time.

- **NLTK (Natural Language Toolkit)**

NLTK complements spaCy by providing essential preprocessing tools. It is used for tasks such as tokenization, stop word removal, stemming, and lemmatization. These processes help clean and normalize text data, ensuring consistency across different resumes. NLTK is

particularly useful for handling basic text operations and preparing data for further analysis. By removing irrelevant information and standardizing text, it improves the performance of subsequent AI models.

- **Sentence-Transformers (Semantic Embedding Models)**

Sentence-Transformers are used to generate vector representations of text. These embeddings capture the semantic meaning of sentences, allowing the system to understand context rather than just keywords. This technology is crucial for comparing resumes with job descriptions. It enables the system to identify similarities even when different words are used to describe the same concept. Sentence-Transformers significantly enhance the accuracy of candidate matching by focusing on meaning rather than exact matches.

- **Cosine Similarity (Similarity Measurement Technique)**

Cosine similarity is used to measure the similarity between vector embeddings. It calculates the angle between two vectors to determine how closely they are related. This technique is widely used in text analysis and recommendation systems. In this project, cosine similarity helps rank candidates based on how well their resumes match the job description. It provides a numerical score that reflects relevance, enabling objective and data-driven decision-making.

- **Large Language Models (LLMs)**

Large Language Models are advanced AI systems capable of understanding and generating human-like text. They are trained on vast datasets and can perform complex language tasks such as summarization, question generation, and content analysis. In the AI Resume Intelligence Hub, LLMs are used to evaluate resumes, provide feedback, and generate insights. They enhance the system's ability to interpret complex information and deliver meaningful recommendations. LLMs also improve user interaction by generating natural and context-aware responses.

- **LangChain (LLM Integration Framework)**

LangChain is used to manage and integrate Large Language Models within the system. It acts as a bridge between the application and AI models, enabling seamless interaction. LangChain supports chaining multiple operations, allowing complex workflows such as resume analysis and feedback generation. It enhances the flexibility of the system by enabling dynamic AI-driven processes. This framework plays a key role in orchestrating different AI components and ensuring smooth execution.

- **Streamlit (Frontend Web Framework)**

Streamlit is used to develop the user interface of the system. It allows developers to create

interactive web applications using Python, eliminating the need for separate frontend technologies. Streamlit provides features such as file upload, real-time updates, and dynamic content rendering. It ensures a smooth and user-friendly experience for both recruiters and candidates. The simplicity of Streamlit enables rapid development and easy maintenance of the interface.

- **SpeechRecognition (Voice Input Technology)**

SpeechRecognition enables the system to capture and process voice input from users. It converts spoken language into text, allowing the system to analyze responses during mock interviews. This technology enhances user interaction by providing a hands-free and natural way of communication. It plays a crucial role in the mock interview module, enabling realistic simulation of interview scenarios.

- **gTTS (Google Text-to-Speech)**

gTTS is used to convert text into speech, enabling the system to communicate with users in an interactive manner. It is particularly useful in the mock interview module, where interview questions are delivered in audio form. This creates a more engaging and realistic experience for users. gTTS supports multiple languages and provides clear and natural-sounding speech output.

- **LaTeX (Document Generation System)**

LaTeX is used for generating professional and ATS-friendly resumes. It provides precise control over document formatting, ensuring high-quality output. LaTeX templates are used to structure resumes in a consistent and visually appealing manner. This technology helps candidates create well-organized resumes that meet industry standards. It also ensures compatibility with Applicant Tracking Systems.

- **PDF Processing Libraries (PyPDF, etc.)**

PDF processing libraries are used to extract text from resume files. These libraries enable the system to read and process PDF documents efficiently. They handle different file formats and structures, ensuring accurate text extraction. This step is essential for converting resumes into a format suitable for analysis. Reliable PDF processing ensures that no important information is lost during extraction.

- **Docker (Containerization and Deployment)**

Docker is used to package the application and its dependencies into containers. This ensures consistent performance across different environments. Docker simplifies deployment by eliminating compatibility issues and enabling easy scaling. It allows the system to run on various platforms without modification. Containerization also improves resource

management and supports efficient handling of large workloads.

- **System Architecture (Modular Design)**

The system follows a modular architecture, where different components operate independently. This design improves flexibility, scalability, and maintainability. Each module, such as resume parsing, semantic matching, and interview simulation, performs a specific function. The modular approach allows easy integration of new features and technologies. It ensures that the system can evolve over time without disrupting existing functionalities.

- **Data Processing Techniques**

Efficient data processing techniques are used to handle large volumes of resumes. These techniques ensure fast and accurate analysis. Data cleaning, normalization, and transformation are essential steps in preparing data for AI models. Efficient algorithms and data structures are used to optimize performance and reduce processing time.

- **Visualization Tools**

Visualization tools are used to present results in an understandable format. Graphs, charts, and dashboards help users interpret data easily. These tools provide insights into candidate rankings, skill distributions, and performance metrics. Visualization enhances decision-making by presenting complex information in a clear and concise manner.

- **Security and Data Handling**

Security measures are implemented to protect sensitive user data. Techniques such as encryption and secure communication protocols ensure data privacy. Access control mechanisms restrict unauthorized access. These measures are essential for maintaining trust and ensuring compliance with data protection standards.

- **Scalability and Performance Optimization**

The system is designed to handle increasing workloads efficiently. Scalability is achieved through containerization and modular design. Performance optimization techniques ensure that the system remains responsive even with large datasets. This makes the platform suitable for real-world applications.

Chapter 6

System Design

The system design of the **AI Resume Intelligence Hub** defines the overall structure, workflow, and interaction between different components of the application. It focuses on how data flows through the system, how modules interact, and how different technologies are integrated to achieve intelligent resume screening and candidate evaluation. The design is developed with key considerations such as scalability, modularity, efficiency, and user experience.

The system is divided into two major parts: **System Architecture** and **Module Description**, which together provide a complete understanding of how the platform operates.

6.1 System Architecture

The system architecture of the AI Resume Intelligence Hub follows a **modular and layered design approach**, ensuring flexibility, scalability, and ease of maintenance. It consists of three main layers:

- **Frontend Layer (User Interface)**
- **Backend Layer (Processing & Logic)**
- **AI & Data Processing Layer**

Each layer performs specific tasks and communicates with others to ensure smooth system operation.

6.1.1 Overview of Architecture

The architecture is designed to handle the complete lifecycle of resume processing, from user input to final output. The process begins when a user uploads a resume and ends with insights such as candidate ranking, feedback, and generated results.

6.1.2 Architectural Components

Frontend Layer

The frontend acts as the interface between the user and the system. It is responsible for:

- Accepting resume uploads
- Displaying results and rankings
- Providing interaction for mock interviews
- Allowing resume generation

This layer is built using **Streamlit**, which provides a simple yet powerful interface for building interactive applications.

Backend Layer

The backend handles:

- Data processing
- Business logic
- API integration
- Communication between modules

It ensures that user requests are processed efficiently and results are generated accurately.

AI Processing Layer

This is the core intelligence layer of the system. It includes:

- NLP processing
- Semantic matching
- Machine learning models
- Large Language Models

This layer transforms raw data into meaningful insights.

6.1.3 Data Flow in the System

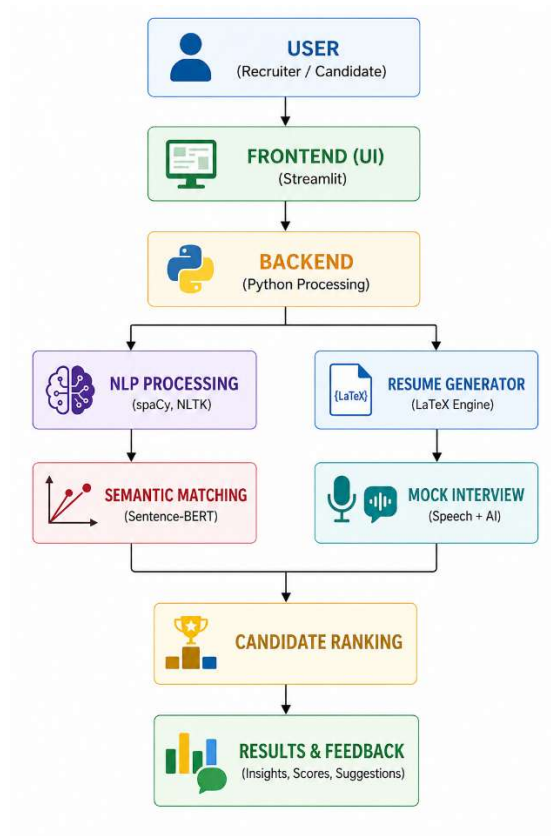
The system follows a structured data flow:

1. User uploads resume

2. Resume is processed and cleaned
3. Key features are extracted
4. Semantic matching is performed
5. Candidates are ranked
6. Results are displayed

This pipeline ensures efficient and accurate processing.

6.1.4 System Architecture Diagram



6.1.5 Key Design Features

- **Modular Design:** Each component works independently
- **Scalability:** Can handle large datasets
- **Flexibility:** Easy to update and expand

- **Efficiency:** Fast processing and response
- **User-Friendly:** Simple interface

6.2 Module Description

The AI Resume Intelligence Hub consists of several modules, each responsible for a specific function. These modules work together to provide a complete recruitment solution.

6.2.1 Resume Upload Module

This module allows users to upload resumes in PDF format. It acts as the entry point of the system.

Key Functions:

- Accept file input
- Validate file format
- Store data temporarily

This module ensures that only valid resumes are processed.

6.2.2 Text Extraction Module

After uploading, resumes are processed to extract text.

Functions:

- Convert PDF to text
- Handle different formats
- Preserve important content

This module ensures accurate extraction of data from resumes.

6.2.3 Data Preprocessing Module

This module cleans and prepares the extracted text.

Functions:

- Remove noise and symbols

- Tokenization
- Stop word removal
- Lemmatization

It ensures data consistency and quality.

6.2.4 Feature Extraction Module

This module identifies important information from resumes.

Extracted Features:

- Skills
- Education
- Experience
- Certifications

It converts unstructured text into structured data.

6.2.5 Semantic Matching Module

This is the core intelligence module.

Functions:

- Convert text into embeddings
- Compare resume with job description
- Calculate similarity score

This ensures accurate candidate-job matching.

6.2.6 Candidate Ranking Module

This module ranks candidates based on relevance.

Functions:

- Assign similarity scores
- Sort candidates
- Provide ranking list

It helps recruiters identify top candidates quickly.

6.2.7 Resume Generator Module

This module helps candidates create professional resumes.

Features:

- Predefined templates
- ATS-friendly format
- Automated formatting

It improves resume quality.

6.2.8 Mock Interview Module

This module simulates real interview scenarios.

Functions:

- Generate questions
- Capture voice input
- Provide feedback

It enhances candidate preparation.

6.2.9 Feedback & Analytics Module

This module provides insights and suggestions.

Features:

- Skill gap analysis
- Performance feedback
- Improvement suggestions

It benefits both recruiters and candidates.

6.2.10 Database / Storage Module

This module manages data storage.

Functions:

- Store resumes
- Save results
- Maintain records

It ensures data persistence.

6.2.11 Integration Module

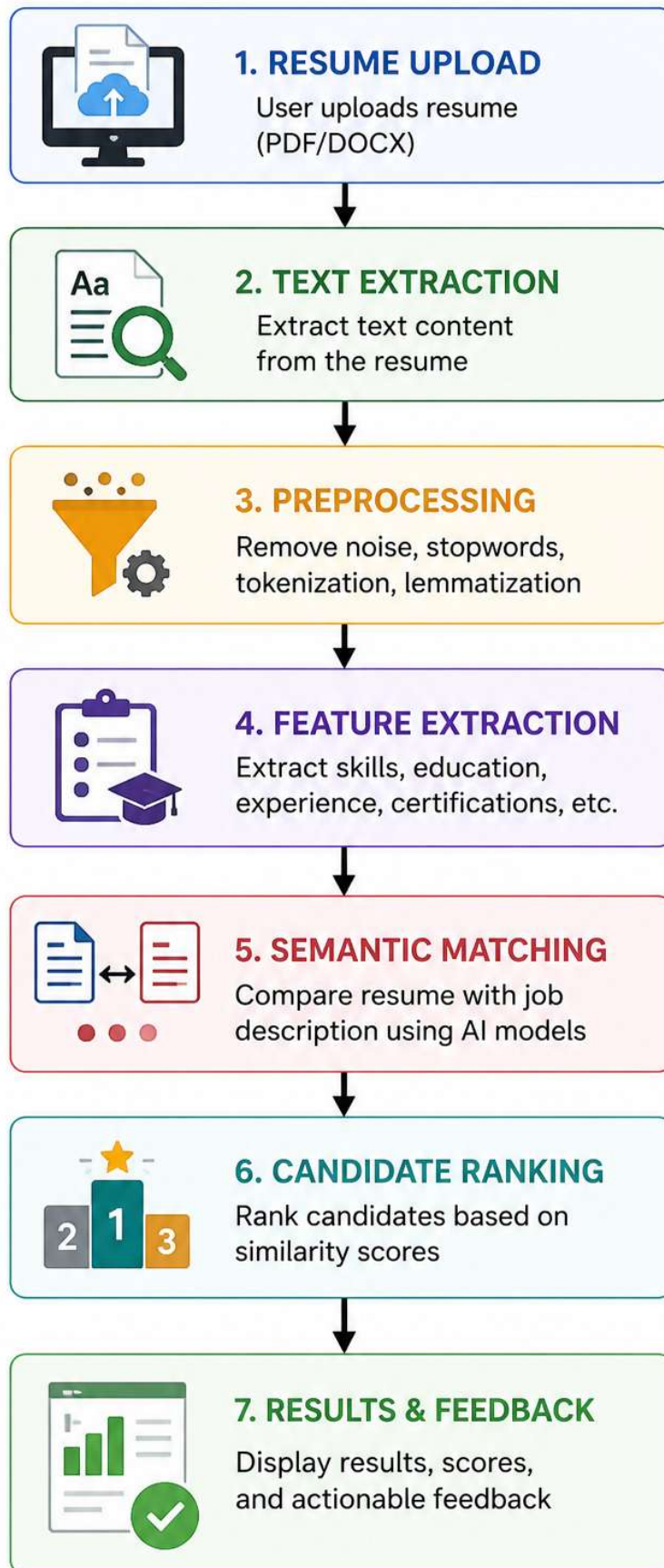
This module connects all components.

Functions:

- Manage communication
- Ensure smooth workflow
- Handle API calls

It acts as the backbone of the system.

6.2.12 Module Interaction Flow



Chapter 7

Implementation

The implementation phase of the AI Resume Intelligence Hub focuses on transforming the conceptual design and methodology into a fully functional system. This phase involves integrating various technologies, developing system modules, and ensuring seamless interaction between components. The implementation is carried out in a structured manner, following a modular approach that allows flexibility, scalability, and efficient performance.

The system is implemented using Python as the core programming language, with various libraries and frameworks integrated to handle specific functionalities such as Natural Language Processing, semantic matching, user interaction, and deployment. The implementation process can be divided into several stages, each responsible for a key part of the system.

7.1 Environment Setup and Configuration

The initial step in implementation involves setting up the development environment. Python is installed along with essential libraries such as spaCy, NLTK, Sentence-Transformers, Streamlit, and other supporting packages. Virtual environments are used to manage dependencies and ensure that the system remains isolated from other applications. Docker is configured to containerize the application, allowing it to run consistently across different environments. This ensures that the system can be deployed easily without compatibility issues. Proper configuration of libraries and tools is crucial for smooth execution of the system.

7.2 Frontend Development

The frontend of the system is developed using Streamlit, which provides a simple yet powerful interface for building interactive web applications. The interface is designed to be user-friendly, allowing both recruiters and candidates to interact with the system easily. Users can upload resumes, input job descriptions, and view results through the interface. The frontend also includes sections for resume generation and mock interviews. Real-time updates and dynamic content rendering are implemented to enhance user experience. The

design focuses on simplicity and clarity, ensuring that users can navigate the system without difficulty.

7.3 Resume Upload and Text Extraction

The implementation begins with the resume upload module, where users can upload resumes in PDF format. Streamlit provides file upload functionality, which is integrated into the system.

Once a resume is uploaded, text extraction is performed using PDF processing libraries.

These libraries read the content of the PDF file and convert it into plain text. The system is designed to handle different resume formats and structures, ensuring accurate extraction of information. Error handling mechanisms are implemented to manage cases where files are corrupted or unsupported.

7.4 Data Preprocessing

After text extraction, the data is preprocessed to improve quality and consistency. This step is implemented using NLP libraries such as NLTK and spaCy.

The preprocessing pipeline includes tokenization, which splits text into individual words or tokens. Stop words are removed to eliminate irrelevant information, and lemmatization is applied to reduce words to their base forms. Special characters, numbers, and unnecessary symbols are also removed.

This step ensures that the data is clean and standardized, making it suitable for further analysis. The implementation focuses on optimizing preprocessing to handle large volumes of data efficiently.

7.5 Feature Extraction

Feature extraction is implemented using NLP techniques to identify key information from resumes. spaCy's Named Entity Recognition is used to detect entities such as skills, organizations, and dates.

Custom rules and patterns are also implemented to extract specific information such as

educational qualifications and work experience. The extracted features are stored in a structured format, such as dictionaries or data frames, enabling easy access and analysis. This module plays a crucial role in converting unstructured text into meaningful data that can be used for candidate evaluation.

7.6 Semantic Matching Implementation

Semantic matching is one of the most important components of the system. It is implemented using Sentence-Transformers, which generate embeddings for both resumes and job descriptions.

The embeddings are numerical representations of text that capture semantic meaning. Cosine similarity is then used to calculate the similarity between embeddings. This results in a similarity score that indicates how closely a resume matches a job description.

The implementation ensures that the system can handle multiple resumes and compare them efficiently with job descriptions. Optimization techniques are applied to reduce computation time and improve performance.

7.7 Candidate Ranking System

Based on the similarity scores, candidates are ranked in order of relevance. The ranking system is implemented using sorting algorithms that arrange candidates based on their scores.

Additional parameters such as experience level and skill importance can be incorporated to refine the ranking process. The results are displayed in a clear and organized format, allowing recruiters to easily identify top candidates.

This module ensures that the recruitment process becomes faster and more efficient.

7.8 Resume Generation Module

The resume generation feature is implemented using LaTeX, which provides high-quality document formatting. Users can input their details through the interface, and the system generates a structured resume using predefined templates.

The implementation includes dynamic content insertion, where user data is mapped to LaTeX templates. The generated document is compiled into a PDF file, which can be downloaded by the user.

This module helps candidates create professional and ATS-friendly resumes, improving their chances of selection.

7.9 Mock Interview Module

The mock interview module is implemented using speech processing technologies.

SpeechRecognition is used to capture audio input from users and convert it into text.

The system generates interview questions using AI models, and gTTS is used to convert these questions into speech. This creates an interactive interview experience.

User responses are analyzed to provide feedback on content and communication skills. This module is designed to simulate real interview scenarios and help candidates improve their performance.

7.10 Feedback and Analytics

The system provides feedback based on the analysis of resumes and interview performance.

This is implemented using AI models that generate insights and suggestions.

Feedback includes skill gap analysis, improvement recommendations, and performance evaluation. Visualization tools are used to present results in the form of charts and graphs.

This module enhances user experience by providing actionable insights.

7.11 Backend Integration

The backend integrates all modules and ensures smooth communication between them. It handles data flow, processing, and result generation.

APIs and function calls are used to connect different components, ensuring that data moves seamlessly through the system. The backend is optimized for performance and reliability.

7.12 Database and Storage

Data storage is implemented to manage resumes, results, and user information. Structured storage systems are used to maintain records and ensure data persistence.

Efficient data handling techniques are applied to manage large volumes of data. Security measures are implemented to protect sensitive information.

7.13 Deployment and Containerization

The system is deployed using Docker, which packages the application and its dependencies into containers. This ensures consistent performance across different environments.

Deployment configurations are set up to enable scalability and efficient resource management. The system can be hosted on cloud platforms, allowing access from different locations.

7.14 Testing and Optimization

The implementation includes extensive testing to ensure system reliability. Unit testing, integration testing, and performance testing are conducted to identify and fix issues.

Optimization techniques are applied to improve speed and accuracy. This includes optimizing algorithms, reducing computation time, and improving resource utilization.

7.15 Security Implementation

Security measures are implemented to protect user data. Encryption techniques are used to secure sensitive information, and access control mechanisms are applied to restrict unauthorized access.

These measures ensure that the system complies with data protection standards and maintains user trust.

7.16 Challenges Faced During Implementation

During implementation, several challenges are encountered, including handling diverse resume formats, ensuring accurate text extraction, and optimizing performance for large

datasets.

These challenges are addressed through advanced algorithms, robust error handling, and continuous testing.

7.17 Final Integration and Execution

The final stage involves integrating all modules and ensuring that the system functions as a unified platform. The workflow is tested end-to-end to verify that each component operates correctly.

The system is then deployed and made accessible to users, providing a complete solution for intelligent resume screening and recruitment.

Chapter 8

Testing and Results

8.1 CODE

```
import streamlit as st
import os
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import sys
import spacy
from typing import List, Dict, Any, Optional
from src.parse_resume import parse_resume, extract_features, load_nlp
from src.rank_candidates import rank_candidates, generate_questions
from src.utils import validate_inputs, save_to_csv
from src.resume_editor import (
    suggest_improvements, generate_html_resume, optimize_summary,
    convert_html_to_pdf, get_templates, save_user_template,
    get_default_templates, delete_user_template, generate_txt_resume,
    generate_latex_resume
)
from src.anonymizer import anonymize_candidate, apply_blind_mode
from src.visualizations import create_skill_network
from src.team_builder import suggest_squad
from src.gauntlet import generate_challenge
from src.job_tracker import render_tracker_ui
from src.job_intelligence import generate_search_links, suggest_roles, fetch_live_jobs, get_company_logo_fallback_api
from src.skill_analysis import (
    keyword_gap_analysis, extract_implicit_skills, detect_bias,
    sanity_check, grade_bullet_points
)
from src.ai_rewriter import rewrite_bullet, rewrite_all_bullets, semantic_job_match_reasoning
from src.ai_rewriter import optimize_summary as ai_optimize_summary
from src.latex_generator import generate_latex, compile_latex_to_pdf as latex_compile_pdf, LATEX_TEMPLATES, is_pdflatex_available
from src.auth import (
    sign_up, sign_in, get_user_display,
    get_oidc_auth_url, process_oidc_callback
)
import io
import base64
import json
from datetime import datetime
import pytz
from gtts import gTTS
import tempfile
import speech_recognition as sr
import src.security as security

st.set_page_config(
    page_title="AI Resume Intelligence Hub | Advanced Screener & Studio",
    page_icon="🧠",
    layout="wide",
    initial_sidebar_state="collapsed"
)
# --- Cache Wrappers ---
@st.cache_data(ttl=3600)
def cached_load_nlp():
    return load_nlp()

@st.cache_data(ttl=300)
def cached_fetch_live_jobs_b64(skills):
    return fetch_live_jobs(skills)
```

```

@st.cache_data(ttl=300)
def cached_generate_search_links(candidate_data):
    return generate_search_links(candidate_data)

@st.cache_data(ttl=3600)
def cached_detect_bias(text):
    return detect_bias(text)

@st.cache_data(ttl=3600)
def cached_keyword_gap_analysis(data, jd, openai_key, hf_token, gemini_key, provider):
    return keyword_gap_analysis(data, jd, openai_key, hf_token, gemini_key, provider)

@st.cache_data(ttl=3600)
def cached_semantic_match(ft, jd, ok, gk, ht, p):
    return semantic_job_match_reasoning(ft, jd, ok, gk, ht, p)

# --- 1. Session State ---
if 'theme' not in st.session_state:
    st.session_state.theme = 'dark'
if 'processed_data' not in st.session_state:
    st.session_state.processed_data = {}
if 'rankings' not in st.session_state:
    st.session_state.rankings = []
if 'selected_template' not in st.session_state:
    st.session_state.selected_template = "Executive Slate"
if 'is_authenticated' not in st.session_state:
    st.session_state.is_authenticated = False
if 'current_page' not in st.session_state:
    st.session_state.current_page = "home"
if 'blind_mode' not in st.session_state:
    st.session_state.blind_mode = False
# AI settings
if 'openai_key' not in st.session_state:
    st.session_state.openai_key = os.environ.get('OPENAI_API_KEY', "")
if 'gemini_key' not in st.session_state:
    st.session_state.gemini_key = os.environ.get('GEMINI_API_KEY', "")
if 'ai_provider' not in st.session_state:
    st.session_state.ai_provider = 'OpenAI'
if 'hf_token' not in st.session_state:
    st.session_state.hf_token = os.environ.get('HF_TOKEN', "")
# Job-specific versioning
if 'resume_versions' not in st.session_state:
    st.session_state.resume_versions = {}
# LaTeX section order per candidate
if 'latex_section_order' not in st.session_state:
    st.session_state.latex_section_order = {}
# selected latex template
if 'selected_latex_template' not in st.session_state:
    st.session_state.selected_latex_template = "Teal Professional"
# Auth state
if 'user_profile' not in st.session_state:
    st.session_state.user_profile = None
if 'is_signed_in' not in st.session_state:
    st.session_state.is_signed_in = False
# Voice transcript
if 'voice_transcript' not in st.session_state:
    st.session_state.voice_transcript = ""
if 'llm_match_result' not in st.session_state:
    st.session_state.llm_match_result = {}

# Constants
LIB_DIR = "data/system_resumes"

if not os.path.exists(LIB_DIR):
    os.makedirs(LIB_DIR)

# --- 2. Page Config ---
# (Handled at top of file)

def render_safe_iframe(html_content, height=180):

```

```

"""Safely renders HTML in an iframe bypassing Streamlit's sandbox bugs while maintaining theme transparency."""
theme = st.session_state.get('theme', 'dark')
bg_color = "#0B0F19" if theme == 'dark' else "#F8FAFC"
text_color = "#E2E8F0" if theme == 'dark' else "#0F172A"
transparent_wrapper = f'<!DOCTYPE html><html><head><meta charset=utf-8><style>body {{ background-color: {bg_color} !important; color:
{text_color} !important; margin: 0; padding: 0; }}</style></head><body>{html_content}</body></html>'
b64_html = base64.b64encode(transparent_wrapper.encode('utf-8')).decode('utf-8')
data_uri = f"data:text/html;base64,{b64_html}"
iframe = f'<iframe src="{data_uri}" style="width:100%; height:{height}px; border:none; border-radius:10px; background: {bg_color};" title="Neural
Visualization"></iframe>'
st.markdown(iframe, unsafe_allow_html=True)

```

--- 3. Ultra-Premium Adaptive Design System ---

```

def apply_adaptive_theme():
    t = st.session_state.theme
    if t == 'dark':
        bg = "#0B0F19"; text = "#E2E8F0"; card = "#151F32"; card_border = "#1F2937"
        accent = "#2DD4BF"; accent_light = "#5EEAD4"; accent_text = "#042F2E"
        success = "#10B981"; warning = "#F59E0B"; error = "#EF4444"
        glass = "rgba(21, 31, 50, 0.75)"
        muted = "#9CA3AF"
        color_scheme = "dark"
    else:
        bg = "#F8FAFC"; text = "#0F172A"; card = "#FFFFFF"; card_border = "#E2E8F0"
        accent = "#0D9488"; accent_light = "#14B8A6"; accent_text = "#FFFFFF"
        success = "#059669"; warning = "#D97706"; error = "#DC2626"
        glass = "rgba(255, 255, 255, 0.85)"
        muted = "#64748B"
        color_scheme = "light"

css = f'"""
<style>
:root {{
  --bg: {bg}; --text: {text}; --accent: {accent}; --card: {card};
  --card-border: {card_border}; --muted: {muted}; --glass: {glass};
  color-scheme: {color_scheme};
}}

.stApp {{
  background: radial-gradient(circle at top right, {accent} 1a, transparent), {bg} !important;
  color: {text} !important;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif !important;
}}

div[data-testid="stButton"] button {{
  display: flex !important;
  align-items: center !important;
  justify-content: center !important;
  gap: 0.4em !important;
}}

[data-testid="stSidebar"] {{ display: none !important; }}

h1, h2, h3, h4, h5, h6, .brand-text {{
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif !important;
  letter-spacing: -0.04em !important;
  color: {text} !important;
  font-weight: 800 !important;
}}

h1 {{ background: linear-gradient(to bottom right, {text}, {muted}); -webkit-background-clip: text; -webkit-text-fill-color: transparent; }}

p, li, label, .stMarkdown, .stText, .stCaption {{
  color: {text}d9 !important;
  font-weight: 400 !important;
}}

.stButton>button, .stDownloadButton>button, .stLinkButton>a {{

```

```

background: linear-gradient(135deg, {accent}, {accent_light}) !important;
color: {accent_text} !important; border: none !important; border-radius: 12px !important;
font-weight: 700 !important; transition: all 0.4s cubic-bezier(0.175, 0.885, 0.32, 1.275) !important;
box-shadow: 0 4px 15px {accent}33;
letter-spacing: -0.01em;
}}

.stButton>button:hover {{
  transform: translateY(-3px) scale(1.02);
  box-shadow: 0 12px 25px {accent}4d !important;
}}

/* Glass Cards */
div[data-testid="stVerticalBlock"] > div[style*="border"] {{
  background: {glass} !important;
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid {card_border} !important;
  border-radius: 20px !important;
  transition: all 0.5s ease !important;
  box-shadow: 0 10px 30px rgba(0,0,0,0.05);
}}

div[data-testid="stVerticalBlock"] > div[style*="border"]:hover {{
  border-color: {accent}55 !important;
  transform: translateY(-5px);
  box-shadow: 0 20px 40px rgba(0,0,0,0.1);
}}

.stTabs [data-baseweb="tab-list"] {{
  border-bottom: 2px solid {card_border};
}}

.stTabs [data-baseweb="tab"] {{
  color: {muted} !important;
  transition: all 0.3s ease !important;
  border-radius: 8px 8px 0 0 !important;
}}

.stTabs [aria-selected="true"] {{
  color: {accent} !important;
  background: {accent}11 !important;
  border-bottom: 4px solid {accent} !important;
}}

/* Metrics */
[data-testid="stMetricValue"] {{ font-weight: 800 !important; color: {accent} !important; font-size: 2.5rem !important; }}
[data-testid="stMetricLabel"] {{ font-weight: 600 !important; text-transform: uppercase; letter-spacing: 0.1em; font-size: 0.8rem !important; }}

.template-card {{
  transition: all 0.3s ease;
  cursor: pointer;
}}
.template-card:hover {{
  transform: scale(1.05);
}}

/* Auth page styling */
.auth-container {{
  max-width: 480px;
  margin: 0 auto;
  padding: 40px;
}}

/* ===== */
/* UNIVERSAL DARK/LIGHT MODE OVERRIDES */
/* ===== */

/* Text Inputs, Text Areas, Number Inputs */
.stTextInput input, .stTextArea textarea, .stNumberInput input {{
  background-color: {card} !important;

```

```

    color: {text} !important;
    border: 1px solid {card_border} !important;
    border-radius: 10px !important;
  }}
.stTextInput input:focus, .stTextArea textarea:focus, .stNumberInput input:focus {{
  border-color: {accent} !important;
  box-shadow: 0 0 2px {accent}33 !important;
}}

/* Labels */
.stTextInput label, .stTextArea label, .stNumberInput label,
.stSelectbox label, .stMultiselect label, .stSlider label,
.stFileUploader label, .stRadio label, .stCheckbox label,
.stToggle label {{
  color: {text} !important;
}}

/* Selectbox / Dropdown */
[data-baseweb="select"] {{
  background-color: {card} !important;
}}
[data-baseweb="select"] > div {{
  background-color: {card} !important;
  color: {text} !important;
  border: 1px solid {card_border} !important;
  border-radius: 10px !important;
}}
[data-baseweb="popover"] {{
  background-color: {card} !important;
}}
[data-baseweb="menu"] {{
  background-color: {card} !important;
}}
[data-baseweb="menu"] li {{
  color: {text} !important;
}}
[data-baseweb="menu"] li:hover {{
  background-color: {accent}22 !important;
}}

/* File Uploader */
[data-testid="stFileUploader"] {{
  background-color: {card} !important;
  border: 1px dashed {card_border} !important;
  border-radius: 15px !important;
}}
[data-testid="stFileUploader"] section {{
  color: {text} !important;
}}

/* Expanders */
[data-testid="stExpander"] {{
  background-color: {glass} !important;
  border: 1px solid {card_border} !important;
  border-radius: 15px !important;
}}
[data-testid="stExpander"] summary {{
  color: {text} !important;
}}
details[data-testid="stExpander"] > div {{
  color: {text} !important;
}}

/* DataFrames / Tables */
[data-testid="stDataFrame"] {{
  background-color: {card} !important;
  border-radius: 10px !important;
}}

/* Slider */
.stSlider [data-baseweb="slider"] div {{

```

```

    color: {text} !important;
  }}
.stSlider [data-testid="stTickBarMin"], .stSlider [data-testid="stTickBarMax"] {{
  color: {muted} !important;
}}

/* Info/Warning/Error/Success Boxes */
[data-testid="stAlert"] {{
  border-radius: 12px !important;
}}

/* Dividers */
hr {{
  border-color: {card_border} !important;
}}

/* Tooltips and helper text */
.stTooltipIcon {{
  color: {muted} !important;
}}

/* Caption text */
.stCaption, [data-testid="stCaptionContainer"] {{
  color: {muted} !important;
}}

/* Status widget */
[data-testid="stStatusWidget"] {{
  background-color: {card} !important;
  color: {text} !important;
}}

/* Responsive Layout Optimization */
@media (max-width: 900px) {{
  h1 {{ font-size: 2.4rem !important; }}
  h2 {{ font-size: 1.8rem !important; }}
  .stTabs [data-baseweb="tab-list"] {{
    flex-wrap: wrap !important;
    gap: 10px !important;
  }}
  .responsive-flex-card {{
    flex-direction: column !important;
    align-items: flex-start !important;
    text-align: left !important;
  }}
  .responsive-flex-card > div:first-child {{
    margin-bottom: 10px;
  }}
}}

@media (max-width: 600px) {{
  h1 {{ font-size: 2.0rem !important; }}
  h2 {{ font-size: 1.5rem !important; }}
  div[data-testid="stVerticalBlock"] > div[style*="border"] {{
    padding: 1.2rem !important; /* Reduce padded glass cards on mobile */
  }}
  [data-testid="stMetricValue"] {{ font-size: 2rem !important; }}
  .stTabs [data-baseweb="tab"] {{ font-size: 0.95rem !important; padding: 10px !important; }}
}}
</style>"""

js = """
<script>
  // Real-time Accessibility & Form Patch for Streamlit
  const patchAccessibility = () => {
    const inputs = window.parent.document.querySelectorAll('input, textarea');
    inputs.forEach(input => {
      // 1. Fix Autocomplete (Avoid 'empty' or missing attributes)
      if (!input.hasAttribute('autocomplete') || input.getAttribute('autocomplete') === "") {
        const labelText = (input.getAttribute('aria-label') || "").toLowerCase();
        if (labelText.includes('email') || labelText.includes('username')) {

```

```

        input.setAttribute('autocomplete', 'username');
    } else if (input.type === 'password' || labelText.includes('password')) {
        input.setAttribute('autocomplete', 'current-password');
    } else {
        input.setAttribute('autocomplete', 'off'); // Default for resume fields
    }
}

// 2. Fix Name/ID for Form Validation & Lighthouse
if (!input.name || input.name.startsWith('st-')) {
    input.name = input.id || 'form_field_' + Math.random().toString(36).substr(2, 5);
}
if (!input.id) {
    input.id = input.name;
}

// 3. Ensure screen reader labels
if (!input.hasAttribute('aria-label') && input.placeholder) {
    input.setAttribute('aria-label', input.placeholder);
}
});
};

// MutationObserver to watch for Streamlit dynamic rendering cycles
const accessibilityObserver = new MutationObserver((mutations) => {
    patchAccessibility();
});

accessibilityObserver.observe(window.parent.document.body, { childList: true, subtree: true });

// Initial execution
setTimeout(patchAccessibility, 500);
</script>
"""

seo = """
<!-- SEO & Accessibility Metadata -->
<div style="display:none;" aria-hidden="true">
    <meta name="description" content="AI Resume Intelligence Hub - Advanced screening, STAR-method analysis, and resume building with live neural
previews.">
    <p>AI-powered resume analysis, STAR-method grading, and professional resume building with live neural previews.</p>
</div>
"""

st.markdown(css + js + seo, unsafe_allow_html=True)

# Auto-implement security layers
security.implement_all()

import urllib.parse
import requests

def compile_latex_to_pdf(latex_code: str) -> bytes:
    try:
        url = "https://latexonline.cc/compile?text=" + urllib.parse.quote(latex_code)
        req = requests.get(url, timeout=45)
        if req.status_code == 200: return req.content
    except Exception: pass
    return b""

# --- 4. Logic & Helpers ---
@st.cache_resource
def get_nlp_model(): return cached_load_nlp()

@st.cache_data(show_spinner=False)
def _cached_process_candidate(content, _nlp):
    text, has_img = parse_resume(content)
    feats = extract_features(text, _nlp)
    return text, has_img, feats

@st.cache_data(show_spinner=False)

```

```

def _cached_rank_candidates(d_map, jd):
    return rank_candidates(d_map, jd, use_deep=True)

def trigger_analysis(files, jd):
    if not validate_inputs(files, jd): return
    with st.status("🌀 Processing...", expanded=True) as status:
        nlp = get_nlp_model()
        d_map = {}
        all_ops = []

        # Only analyze the explicitly uploaded files
        if files:
            for f in files: all_ops.append((f.name, f.getvalue()))

        if not all_ops:
            st.error("⚠️ No resumes found. Please upload at least one PDF.")
            status.update(label="Scanning Failed", state="error"); return

        for name_orig, content in all_ops:
            name_file = name_orig.replace(".pdf", "")
            text, has_img, feats = _cached_process_candidate(content, nlp)
            feats['filename'] = name_file
            feats['name'] = feats.get('name') or name_file
            feats['has_original_photo'] = has_img
            feats['full_text'] = text # to ensure text is saved
            d_map[name_file] = feats
        st.session_state.processed_data = d_map
        rankings = _cached_rank_candidates(d_map, jd)
        st.session_state.rankings = rankings
        st.session_state.jd_text = jd

        # Save to history if user is logged in
        if st.session_state.get('is_signed_in') and 'user_profile' in st.session_state:
            email = st.session_state.user_profile.get('email')
            if email:
                try:
                    from src.history import save_user_analysis
                    for candidate, score, features in rankings:
                        save_user_analysis(email, jd, candidate, features.get('ats_score_percentage', int(score*100)), features.get('ats_validation', 'Analyzed'))
                except ImportError:
                    pass

        status.update(label="Scanning Complete", state="complete")
        st.rerun()

def generate_natalia_response(user_message: str, chat_history: list) -> str:
    """Generate Natalia's interview response using configured AI or fallback."""
    # 1. Moderation Check (Keyword based)
    rude_keywords = ["stupid", "idiot", "dumb", "hate", "shut up", "fuck", "shit", "bitch", "ass", "bastard"]
    msg_low = user_message.lower()
    if any(k in msg_low for k in rude_keywords):
        return "Excuse me, but there's no need for that. You are being rude, and we can't continue this session in an unprofessional manner. Please maintain a respectful tone for this interview."

    from src.ai_rewriter import _get_llm
    llm = _get_llm(
        st.session_state.openai_key,
        st.session_state.hf_token,
        st.session_state.gemini_key,
        st.session_state.ai_provider
    )
    if llm:
        try:
            from langchain_core.prompts import PromptTemplate
            history_str = "\n".join([f'{m["role"]}.upper(): {m["content"]}' for m in chat_history[-6:]])
            prompt = PromptTemplate.from_template(
                "You are Natalia, a professional HR interviewer conducting a mock interview. "
                "You are warm, encouraging, but also thorough. Your job is to assess the candidate's professional demeanor. "
                "IF the candidate is being offensive, rude, or unprofessional, you MUST politely state that you cannot continue and ask them to be respectful.\n\n"
                "Chat history:\n{history}\n\n"
                "Candidate's latest answer: {answer}\n\n"
            )

```

```

        "Natalia's response:"
    )
    from langchain_core.output_parsers import StrOutputParser
    chain = prompt | llm | StrOutputParser()
    result = chain.invoke({"history": history_str, "answer": user_message})
    if result and len(result.strip()) > 20:
        return result.strip()
    except Exception:
        pass

# Fallback responses
import random
fallbacks = [
    "That's a thoughtful answer. Can you elaborate on the measurable outcomes you achieved? Quantifying your impact helps demonstrate your value.",
    "Great point! How did you handle challenges or pushback during that project? I'd love to hear about your problem-solving approach.",
    "Interesting perspective. Could you walk me through how you collaborated with cross-functional teams? Communication skills are key for this role.",
    "I appreciate that detail. What tools or methodologies did you use, and how did they contribute to the project's success?",
    "Excellent. Let's shift gears — tell me about a time you had to learn something new under a tight deadline. How did you approach it?",
    "That shows great initiative. How do you stay updated with the latest trends in your field? Continuous learning is vital in tech.",
]
return random.choice(fallbacks)

# --- 5. UI Components ---
def render_top_nav(accent):
    with st.container():
        nav_col1, _, nav_col3 = st.columns([6, 2.5, 1.5])
        with nav_col1:
            if st.button("🔵 AI RESUME IQ", key="logo_home", type="secondary"):
                st.session_state.current_page = "home"; st.rerun()
        with nav_col3:
            if st.session_state.is_signed_in:
                c1, c2, c3, c4 = st.columns(4)
            else:
                c1, c2, c3 = st.columns(3)

            if c1.button("🌞", help="Toggle Theme", use_container_width=True):
                st.session_state.theme = "light" if st.session_state.theme == "dark" else "dark"; st.rerun()
            mode_icon = "👁️" if st.session_state.blind_mode else "👁️"
            if c2.button(mode_icon, help="Toggle Blind Mode", use_container_width=True):
                st.session_state.blind_mode = not st.session_state.blind_mode; st.rerun()
            if c3.button("🔑", help="User/Admin login", use_container_width=True):
                st.session_state.current_page = "admin"; st.rerun()

            if st.session_state.is_signed_in:
                if c4.button("🚪", help="Sign Out", use_container_width=True):
                    st.session_state.is_signed_in = False
                    st.session_state.user_profile = None
                    st.session_state.current_page = "home"
                    st.rerun()
    st.divider()

def render_auth_page(accent):
    """User/Admin Sign-in page."""
    st.markdown(f"""<div style='text-align:center; padding:40px 0 20px 0;'>
        <h1 style='font-size:3.5rem !important;'>🔒 System Access</h1>
        <p style='font-size:1.2rem !important; opacity:0.7;'>Login to proceed</p>
    </div>""", unsafe_allow_html=True)

    col1, col2, col3 = st.columns([1, 2, 1])
    with col2:
        with st.container(border=True):
            st.markdown("### Authenticate")
            login_email = st.text_input("Username/Email", key="login_email", placeholder="Enter username")
            login_pass = st.text_input("Password", type="password", key="login_pass", placeholder="Enter password")
            if st.button("🔑 Access Portal", type="primary", use_container_width=True, key="btn_signin"):
                if login_email and login_pass:
                    result = sign_in(login_email, login_pass)
                    if result["success"]:
                        st.session_state.is_signed_in = True
                        st.session_state.user_profile = result["user"]

```

```

        st.toast(f"Welcome back, {result['user']['name']}! 🎉")
        # Route based on role
        if result["user"].get("role") in ("admin", "co-admin"):
            st.session_state.current_page = "admin"
        else:
            st.session_state.current_page = "user_dashboard"
            st.rerun()
        else:
            st.error(result["message"])
    else:
        st.warning("Please enter credentials.")
    if st.button("Cancel", use_container_width=True):
        st.session_state.current_page = "home"
        st.rerun()

def render_user_dashboard(accent):
    """User-specific dashboard with profile info and resume analysis history."""
    profile = st.session_state.user_profile or {}
    st.markdown(f"### 👤 User Dashboard: {profile.get('name', 'User')}")

    dash_tabs = st.tabs(["👤 My Profile", "📄 My Resume History"])

    with dash_tabs[0]:
        col1, col2 = st.columns([1, 2])
        with col1:
            with st.container(border=True):
                st.markdown("### Information")
                st.write(f"***Email:** {profile.get('email')}")
                st.write(f"***Role:** {profile.get('role', 'Member').capitalize()}")
                st.write(f"***Member Since:** {profile.get('created_at', 'N/A')[:10]}")

        with col2:
            with st.container(border=True):
                st.markdown("### Mission Parameters")
                st.info("Your assigned roles and features are managed by your administrator.")
                st.write("Current access level: **Standard Application Access**")
                st.write("---")
                st.write("You may now use all the application features like **Resume Studio**, **Neural analysis**, and **Mock interviews** with your personalized profiles.")

    with dash_tabs[1]:
        st.markdown("### 📄 Your Past Resume Analyses")
        st.caption("Every resume scan you run while signed in is automatically saved here.")

    try:
        from src.history import get_user_history, delete_user_history_entry
        email = profile.get('email', "")
        history = get_user_history(email)

        if not history:
            st.info("No analysis history found yet. Run a resume scan from the home page while signed in!")
        else:
            for idx, entry in enumerate(reversed(history)):
                with st.container(border=True):
                    h_col1, h_col2, h_col3 = st.columns([3, 1, 1])
                    with h_col1:
                        st.markdown(f"***📄 {entry['resume_name']}***")
                        st.caption(f"🕒 {entry['timestamp']} • JD: {_entry.get('job_description_snippet', 'N/A')}")
                    with h_col2:
                        score = entry.get('match_score', 0)
                        color = "#22c55e" if score >= 70 else "#facc15" if score >= 45 else "#ef4444"
                        st.markdown(f"<div style='text-align:center;'><span style='font-size:1.8rem; font-weight:800; color:{color};'>{score}%</span><br><span style='font-size:0.8rem; opacity:0.7;'>{entry.get('ats_label', '')}</span></div>", unsafe_allow_html=True)
                    with h_col3:
                        if st.button("🗑️ Delete", key=f"del_hist_{idx}"):
                            delete_user_history_entry(email, entry['id'])
                            st.rerun()
    except ImportError:
        st.error("History module not available.")

```

```

if st.button("Return to Intelligence Hub", use_container_width=True):
    st.session_state.current_page = "home"; st.rerun()

def render_admin_page(accent):
    st.markdown("## 🛡️ Admin Command Center")
    user_role = st.session_state.user_profile.get('role', "") if st.session_state.user_profile else ""
    is_main = st.session_state.user_profile.get('is_main_admin', False) if st.session_state.user_profile else False

    if user_role not in ('admin', 'co-admin'):
        st.error("Access Denied. Admins and Co-Admins only.")
        if st.button("Return Home"):
            st.session_state.current_page = "home"; st.rerun()
        return

    admin_tabs = st.tabs(["🔧 API Settings", "👤 User Management", "🔄 Change Roles", "⚙️ System"])

    # --- TAB 1: API Settings (admin only) ---
    with admin_tabs[0]:
        if user_role == 'co-admin':
            st.warning("🛡️ API Settings are restricted to Admin accounts.")
        else:
            col1, col2 = st.columns(2)
            with col1:
                st.subheader("🌐 Global API Settings")
                st.session_state.ai_provider = st.selectbox("Primary AI Provider", ["OpenAI", "Gemini", "HuggingFace"])
                if st.session_state.ai_provider == "OpenAI":
                    st.session_state.openai_key = st.text_input("OpenAI Key", value=st.session_state.openai_key, type="password")
                elif st.session_state.ai_provider == "Gemini":
                    st.session_state.gemini_key = st.text_input("Gemini Key", value=st.session_state.gemini_key, type="password")

                st.divider()
                st.subheader("✚ Custom API Add-on")
                custom_name = st.text_input("Custom Provider Name", placeholder="e.g. Anthropic, DeepSeek")
                custom_key = st.text_input(f"{custom_name if custom_name else 'Custom'} API Key", type="password")
                if st.button("Register Custom Key"):
                    st.toast(f"Registered {custom_name} key successfully!")

            with col2:
                st.subheader("⚙️ Mission Parameters")
                st.session_state.pdf_limit = st.slider("PDF Upload Limit per Session", 1, 50, st.session_state.get('pdf_limit', 5))

                st.divider()
                if st.button("🗑️ PURGE SYSTEM CACHE", type="primary", use_container_width=True):
                    st.session_state.processed_data = {}; st.session_state.rankings = []; st.cache_resource.clear(); st.rerun()

    # --- TAB 2: User Management ---
    with admin_tabs[1]:
        st.subheader("👤 User Account Management")
        from src.auth import get_all_users, reset_password, delete_user, sign_up
        all_users = get_all_users()

        # User Table
        df_users = pd.DataFrame(all_users)
        if not df_users.empty:
            st.dataframe(df_users[["name", "email", "role", "created_at"]], use_container_width=True)

        st.divider()
        m_col1, m_col2 = st.columns(2)

        with m_col1:
            st.markdown("#### ✚ Create New Account")
            if not is_main:
                st.warning("Only the Main Admin can create accounts.")
            else:
                new_name = st.text_input("Name", key="new_u_name")
                new_email = st.text_input("Email", key="new_u_email")
                new_pass = st.text_input("Temp Password", type="password", key="new_u_pass")
                new_role = st.selectbox("Role", ["admin", "co-admin", "user"], key="new_u_role")
                if st.button("Create Account"):

```

```

        if new_name and new_email and new_pass:
            res = sign_up(new_name, new_email, "0000000000", new_pass, role=new_role)
            if res["success"]: st.success(f"Account for {new_name} ({new_role}) created!")
            else: st.error(res["message"])
        else: st.warning("Fill all fields")

with m_col2:
    st.markdown("### 🗝️ Reset User Password")
    target_email = st.selectbox("Select User", [u["email"] for u in all_users])
    new_pass_field = st.text_input("New Password", type="password", key="reset_pass_field")
    if st.button("Reset Password"):
        if reset_password(target_email, new_pass_field):
            st.success(f"Password for {target_email} updated!")
        else: st.error("Failed to update password.")

    st.divider()
    st.markdown("### 🗑️ Remove User")
    if user_role == 'co-admin':
        st.warning("🚫 Co-Admins cannot delete users.")
    else:
        removable = [u["email"] for u in all_users if u["email"] != st.session_state.user_profile["email"]]
        if removable:
            del_email = st.selectbox("Select User to Remove", removable)
            if st.button("Confirm Deletion", type="primary"):
                if delete_user(del_email):
                    st.success(f"User {del_email} removed.")
                    st.rerun()
                else: st.error("Could not delete user (might be main admin).")
            else:
                st.caption("No removable users.")

# --- TAB 3: Change Roles (Main Admin only) ---
with admin_tabs[2]:
    st.subheader("👤 Change User Roles")
    if not is_main:
        st.warning("🔒 Only the Main Admin can change user roles.")
    else:
        from src.auth import get_all_users, update_user_role
        all_users_fresh = get_all_users()
        changeable = [u for u in all_users_fresh if u["email"] != st.session_state.user_profile["email"] and not u.get("is_main_admin")]

        if not changeable:
            st.info("No users available to change roles for.")
        else:
            for u in changeable:
                with st.container(border=True):
                    rc1, rc2, rc3 = st.columns([2, 1, 1])
                    with rc1:
                        st.markdown(f"*** {u['name']} ** — ` {u['email']} `")
                        st.caption(f"Current role: **{u.get('role', 'user')}**")
                    with rc2:
                        new_r = st.selectbox("New Role", ["admin", "co-admin", "user"], index=["admin", "co-admin", "user"].index(u.get('role', 'user'))),
                        key=f"role_sel_{u['email']}")
                    with rc3:
                        st.write("") # spacer
                        if st.button("Apply", key=f"role_btn_{u['email']}"):
                            if update_user_role(u['email'], new_r):
                                st.success(f"Role for {u['name']} changed to {new_r}!")
                                st.rerun()
                            else:
                                st.error("Failed to update role.")

# --- TAB 4: System ---
with admin_tabs[3]:
    st.subheader("🏠 System Health")
    st.info("System operational. All neural cores online.")

def render_home_page(accent):
    tab_titles = ["🏠 Home", "📊 Analytics", "🚀 AI Intelligence", "📄 Resume Studio", "🔗 LaTeX Hub", "🔍 AI Review", "💼 Job Intelligence", "🌐"]

```

```

Mock Interview", " 📄 Feedback"]
main_tabs = st.tabs(tab_titles)

# =====
# TAB 1: DASHBOARD
# =====

with main_tabs[0]:
    st.markdown(f"<div style='background: {accent}11; padding:40px; border-radius:24px; border:1px solid {accent}33; margin-bottom:30px;'><h1>Resume
Intelligence Hub</h1><p style='font-size:1.2rem !important; opacity:0.8;'>Precision scanning for the next generation of talent.</p></div>",
unsafe_allow_html=True)
    assets_len = len(st.session_state.processed_data)
    st.markdown(f'"""
<div style="display: flex; gap: 20px; margin-bottom: 20px; flex-wrap: wrap;">
  <div style="flex: 1; padding: 20px; background: {accent}11; border: 1px solid {accent}33; border-radius: 16px;">
    <p style="margin: 0; font-size: 0.85rem; text-transform: uppercase; letter-spacing: 1px; font-weight: 700; color: {accent};">Assets Analyzed</p>
    <div style="font-size: 2.8rem; font-weight: 800; color: {accent}; line-height: 1.2;">{assets_len}</div>
  </div>
  <div style="flex: 1; padding: 20px; background: {accent}11; border: 1px solid {accent}33; border-radius: 16px;">
    <p style="margin: 0; font-size: 0.85rem; text-transform: uppercase; letter-spacing: 1px; font-weight: 700; color: {accent};">Processing Latency</p>
    <div style="font-size: 2.8rem; font-weight: 800; color: {accent}; line-height: 1.2;">0.42s</div>
  </div>
  <div style="flex: 1; padding: 20px; background: {accent}11; border: 1px solid {accent}33; border-radius: 16px;">
    <p style="margin: 0; font-size: 0.85rem; text-transform: uppercase; letter-spacing: 1px; font-weight: 700; color: {accent};">Neural Fidelity</p>
    <div style="font-size: 2.8rem; font-weight: 800; color: {accent}; line-height: 1.2;">99.8%</div>
  </div>
</div>
'''", unsafe_allow_html=True)

    with st.container(border=True):
        st.subheader("🎯 Objective")
        jd = st.text_area("Job Description / Role Requirements", height=120, placeholder="Paste the job description here for deep semantic alignment...")

        st.divider()

        st.subheader("📁 Intake")
        up = st.file_uploader("Upload Candidate PDFs", type=["pdf"], accept_multiple_files=True, help=f"Limit: {st.session_state.get('pdf_limit', 5)} files per
batch")

        st.write("") # spacer
        if st.button("🚀 EXECUTE NEURAL SCAN", use_container_width=True, type="primary"):
            trigger_analysis(up, jd)

    if st.session_state.rankings:
        st.divider(); st.subheader("📊 Match Distribution Analysis")
        df = pd.DataFrame([{'Candidate': r[0], 'Score': r[2].get('ats_score_percentage', min(int(r[1]*100), 100))} for r in st.session_state.rankings])

        # Traffic-light color scale: red(low) → yellow(mid) → green(high)
        is_dark = st.session_state.theme == 'dark'

        # Assign individual bar colors based on score
        def score_to_color(score):
            """Map score to a color: red(0) → orange(30) → yellow(50) → green(80+)"""
            if score >= 80:
                return "rgb(34, 197, 94)" # Green
            elif score >= 60:
                return "rgb(132, 204, 22)" # Lime-green
            elif score >= 45:
                return "rgb(250, 204, 21)" # Yellow
            elif score >= 30:
                return "rgb(251, 146, 60)" # Orange
            else:
                return "rgb(239, 68, 68)" # Red

        bar_colors = [score_to_color(s) for s in df['Score']]

        text_color = '#E2E8F0' if is_dark else '#1E293B'
        grid_color = 'rgba(255,255,255,0.08)' if is_dark else 'rgba(15,23,42,0.08)'
        bg_color = 'rgba(0,0,0,0)' if is_dark else 'rgba(248,250,252,0.5)'

        fig = go.Figure(data=[

```

```

go.Bar(
    x=df['Candidate'],
    y=df['Score'],
    marker_color=bar_colors,
    marker_line=dict(color='rgba(255,255,255,0.3)' if is_dark else 'rgba(0,0,0,0.1)', width=1),
    text=[f'{s:.0f}' for s in df['Score']],
    textposition='outside',
    textfont=dict(color=text_color, size=15, family="-apple-system, system-ui, sans-serif"),
    hovertemplate='<b>%{x}</b><br>Score: %{y:.1f}<extra></extra>',
)
])

fig.update_layout(
    paper_bgcolor='rgba(0,0,0,0)',
    plot_bgcolor=bg_color,
    font_family="-apple-system, system-ui, sans-serif",
    font_color=text_color,
    xaxis=dict(
        title=dict(text="Candidate", font=dict(color=text_color, size=13)),
        color=text_color,
        gridcolor=grid_color,
        tickfont=dict(color=text_color, size=12),
        linecolor=grid_color,
    ),
    yaxis=dict(
        title=dict(text="Match Score", font=dict(color=text_color, size=13)),
        color=text_color,
        gridcolor=grid_color,
        tickfont=dict(color=text_color, size=12),
        linecolor=grid_color,
        range=[0, 115], # Increased range to guarantee text overhead space
    ),
    height=550,
    margin=dict(t=50, b=50, l=60, r=40),
    bargap=0.2,
    hovermode='x unified'
)

fig.add_hline(y=80, line_dash="dash", line_color="rgba(34, 197, 94, 0.6)", annotation_text="Excellent (80-100)", annotation_position="top right",
annotation_font_color=text_color)
fig.add_hline(y=60, line_dash="dash", line_color="rgba(250, 204, 21, 0.6)", annotation_text="Acceptable (60-79)", annotation_position="top right",
annotation_font_color=text_color)
fig.add_hline(y=30, line_dash="dash", line_color="rgba(239, 68, 68, 0.6)", annotation_text="Poor (<30)", annotation_position="top right",
annotation_font_color=text_color)

import plotly.io as pio
html_str = pio.to_html(fig, include_plotlyjs='cdn', full_html=False, config={'responsive': True})
canvas_patch = "<script>const _ogc = HTMLCanvasElement.prototype.getContext; HTMLCanvasElement.prototype.getContext = function(t, a) { if (t
==== '2d') { a = a || {}; a.willReadFrequently = true; } return _ogc.call(this, t, a); };</script>"
render_safe_iframe(canvas_patch + html_str, height=550)

# =====
# TAB 2: ANALYTICS
# =====

with main_tabs[1]:
    st.header("🌌 Precision Galaxy")
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:
        with st.container():
            fig = create_skill_network(st.session_state.processed_data, theme=st.session_state.theme)
            # Reduced height so it doesn't push down the page excessively
            fig.update_layout(height=450, margin=dict(l=0, r=0, b=0, t=20))
            import plotly.io as pio
            html_str = pio.to_html(fig, include_plotlyjs='cdn', full_html=False, config={'responsive': True, 'scrollZoom': True})
            canvas_patch = "<script>const _ogc = HTMLCanvasElement.prototype.getContext; HTMLCanvasElement.prototype.getContext = function(t, a) { if
(t === '2d') { a = a || {}; a.willReadFrequently = true; } return _ogc.call(this, t, a); };</script>"
            render_safe_iframe(canvas_patch + html_str, height=450)
            st.caption("🔍 Interactive 3D Skill Network - Rotate and zoom to explore neural connections.")

# =====

```

```

# TAB 3: AI INTELLIGENCE
# =====
with main_tabs[2]:
    st.markdown("<br>", unsafe_allow_html=True)
    col_h1, col_h2 = st.columns([0.7, 0.3])
    with col_h1:
        st.header("🧠 Neural DNA Analysis")
    with col_h2:
        if st.button("🔄 Refresh Job Links", use_container_width=True):
            st.session_state.live_jobs = None; st.rerun()

    st.caption("Deep decomposition of candidate attributes and semantic alignment.")
    st.markdown("<br>", unsafe_allow_html=True)
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:
        sel = st.selectbox("Select Asset:", list(st.session_state.processed_data.keys()))
        data = st.session_state.processed_data[sel]
        atabs = st.tabs(["📊 Gap Analysis", "🔍 Implicit Skills"])

    with atabs[0]:
        jd_text = st.session_state.get('jd_text', "")
        if not jd_text:
            st.warning("⚠️ No job description found. Run a scan first to enable Gap Analysis.")
        else:
            with st.spinner("Analyzing neural gaps..."):
                gap = cached_keyword_gap_analysis(
                    data, jd_text,
                    openai_key=st.session_state.openai_key,
                    hf_token=st.session_state.hf_token,
                    gemini_key=st.session_state.gemini_key,
                    provider=st.session_state.ai_provider
                )

            if not gap.get('found') and not gap.get('missing'):
                st.error("❌ Neural Analysis failed to extract requirements. Check JD quality.")
                st.info("Ensure the Job Description contains clear skill requirements.")

            f_col, m_col = st.columns(2)
            with f_col:
                st.write("### ✅ Met Requirements")
                found_items = gap.get('found', [])
                if found_items:
                    for item in found_items:
                        st.markdown(f"<div style='background:rgba(34, 197, 94, 0.1); padding:10px; border-radius:8px; margin-bottom:8px; border-left:4px solid #22c55e;'>✅ <b>{item['area']}</b> — {item['evidence']}</div>", unsafe_allow_html=True)
                else:
                    st.caption("No clear requirements met based on the current scan.")
            with m_col:
                st.write("### ❌ Missing Gaps")
                missing_items = gap.get('missing', [])
                if missing_items:
                    for area in missing_items:
                        st.markdown(f"🔴 {area}")
                else:
                    st.success("No critical gaps detected!")

            # Show suggestions
            suggestions = gap.get('suggestions', [])
            if suggestions:
                st.divider()
                st.write("### 💡 AI Suggestions to Bridge Gaps")
                for idx, s in enumerate(suggestions):
                    st.info(f"{idx+1}. {s}")

    with atabs[1]:
        implicit = extract_implicit_skills(data.get('skills', []))
        if implicit:
            st.write("**Implicit Capabilities Detected:**")
            cols = st.columns(3)

```

```

        for i, skill in enumerate(implicit):
            with cols[i % 3]:
                st.markdown(f"🔗 {skill}")
    else:
        st.caption("No implicit skills detected from the current skill set.")

# =====
# TAB 4: RESUME STUDIO
# =====

with main_tabs[3]:
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:
        sel_s = st.selectbox("Target Asset for Studio:", list(st.session_state.processed_data.keys()), key="studio_v3")
        data_s = st.session_state.processed_data[sel_s]

        # ATS/Photo toggle + template gallery toggle
        toggle_col1, toggle_col2 = st.columns(2)
        with toggle_col1:
            is_photo_mode = st.toggle("📷 Photo Template", value=False, key="photo_toggle")
        with toggle_col2:
            show_gallery = st.toggle("🖼️ Show Template Gallery", value=False, key="gallery_toggle")

        if show_gallery:
            st.markdown("### 🏆 Premium Template Selection")
            templates = get_templates()

            categories = {
                "🧠 Intelligence Favorites": ["Executive Slate", "Aura Elite", "Spectrum Pro", "Midnight Pro", "Azure Tech"],
                "📄 Modern & ATS-Friendly": ["Harvard Classic", "ATS Titan", "Glacier Simple", "Arctic Blue", "Minimalist Slate", "Silicon Emerald"],
                "🎨 Creative & Premium": ["Creative Royal", "Visionary Card", "Creative Grid", "Infographic Flow", "Golden Executive", "Nordic Clean"]
            }

            for cat_name, t_list in categories.items():
                # Pre-filter to see if this category should even be shown
                filtered_list = []
                for t_name in t_list:
                    if t_name in templates:
                        t_info = templates[t_name]
                        has_photo = t_info.get('has_photo', False)
                        # Logic: If photo mode is ON, only show photo templates.
                        # If photo mode is OFF, only show text templates.
                        if is_photo_mode:
                            if has_photo: filtered_list.append(t_name)
                        else:
                            if not has_photo: filtered_list.append(t_name)

                if not filtered_list:
                    continue # Skip empty category expanders completely

            with st.expander(f"{cat_name} ({len(filtered_list)})", expanded=(cat_name == "🧠 Intelligence Favorites")):
                # Display in grid of 4
                for i in range(0, len(filtered_list), 4):
                    cols = st.columns(4)
                    for j in range(4):
                        if i + j < len(filtered_list):
                            t_name = filtered_list[i + j]
                            t_info = templates[t_name]
                            with cols[j]:
                                with st.container(border=True):
                                    # show a mini preview of the template instead of icon
                                    thumb_html = generate_html_resume(data_s, t_name, thumbnail=True)
                                    render_safe_iframe(thumb_html, height=180)
                                    st.markdown(f"***{t_name}***")
                                    badge = "ATS-Friendly" if not t_info.get('has_photo', False) else "With Photo"
                                    st.caption(f"{t_info.get('desc', 'Professional')} • {badge}")
                                    if st.button(f"Select {t_name}", key=f"btn_{t_name}", use_container_width=True):
                                        st.session_state.selected_template = t_name; st.toast(f"Switched to {t_name}"); st.rerun()

st.divider()

```

```

else:
    st.caption(f"📄 Current Template: **{st.session_state.selected_template}** — Toggle gallery above to switch.")

# Preview + Editor
col_pre, col_edt = st.columns([1.5, 1], gap="large")
with col_pre:
    st.subheader("📺 Live Preview")
    clean_resume_html = generate_html_resume(data_s, st.session_state.selected_template)
    render_safe_iframe(clean_resume_html, height=900)

with col_edt:
    st.markdown("### ✎ Full Editor")

# --- Profile Photo (Conditional) ---
t_config = get_templates().get(st.session_state.selected_template, {})
if t_config.get('has_photo', False):
    with st.expander("📷 Profile Photo", expanded=True):
        uploaded_photo = st.file_uploader("Upload Profile Image", type=['png', 'jpg', 'jpeg'])
        if uploaded_photo:
            b64 = base64.b64encode(uploaded_photo.getvalue()).decode()
            data_s['photo'] = f"data:{uploaded_photo.type};base64,{b64}"

        if data_s.get('photo'):
            st.markdown(f"<img src='{data_s['photo']}' width='100' style='border-radius:10px; margin-bottom:5px;'>", unsafe_allow_html=True)
            st.caption("Current Photo")
            if st.button("Remove Photo", key="btn_rm_photo"):
                data_s['photo'] = None
                st.rerun()

# --- Section Management ---
with st.expander("📁 Section Management", expanded=False):
    st.caption("Reorder or rename your resume sections.")

    # 1. Section Order
    all_possible = ['summary', 'experience', 'education', 'skills']
    custom_sects = data_s.get('custom_sections', {})
    all_possible += list(custom_sects.keys())

    new_order = st.multiselect("Active Section Order",
                                options=all_possible,
                                default=data_s.get('section_order', ['summary', 'experience', 'education', 'skills']),
                                help="Drag or select sections in the order you want them to appear.")
    data_s['section_order'] = new_order

    st.divider()
    st.markdown("*** ➕ Custom Section***")
    c_name = st.text_input("New Section Name", placeholder="e.g. Projects, Certifications")
    c_val = st.text_area("Section Content (Markdown/Text)", placeholder="Describe your achievements...")
    if st.button("Add Section"):
        if c_name:
            if 'custom_sections' not in data_s: data_s['custom_sections'] = {}
            data_s['custom_sections'][c_name] = c_val
            if c_name not in data_s['section_order']:
                data_s['section_order'].append(c_name)
            st.rerun()

    if custom_sects:
        st.markdown("*** 🗑 Remove Custom Sections***")
        for cs in list(custom_sects.keys()):
            if st.button(f"Delete {cs}", key=f"del_{cs}"):
                del data_s['custom_sections'][cs]
                if cs in data_s['section_order']:
                    data_s['section_order'].remove(cs)
                st.rerun()

# --- Contact Info ---
with st.expander("📧 Contact Information", expanded=True):
    data_s['name'] = st.text_input("Full Name", data_s.get('name', ''), key="ed_name")
    data_s['email'] = st.text_input("Email", data_s.get('email', ''), key="ed_email")

```

```

data_s['phone'] = st.text_input("Phone", data_s.get('phone', ""), key="ed_phone")
data_s['linkedin'] = st.text_input("LinkedIn URL", data_s.get('linkedin', ""), key="ed_linkedin")

# --- Summary ---
with st.expander("📄 Professional Summary", expanded=True):
    heading_sum = data_s.get('original_headings', {}).get('summary', 'Professional Summary')
    new_h_sum = st.text_input("Section Heading", heading_sum, key="ed_h_sum")
    if 'original_headings' not in data_s:
        data_s['original_headings'] = {}
    data_s['original_headings']['summary'] = new_h_sum
    data_s['summary'] = st.text_area("Summary Content", data_s.get('summary', ""), height=100, key="ed_summary")

# --- Experience ---
with st.expander("📄 Experience", expanded=True):
    heading_exp = data_s.get('original_headings', {}).get('experience', 'Professional Experience')
    new_h_exp = st.text_input("Section Heading", heading_exp, key="ed_h_exp")
    data_s['original_headings']['experience'] = new_h_exp

    experience = data_s.get('experience', [])
    updated_experience = []

    for exp_idx, exp_item in enumerate(experience):
        if isinstance(exp_item, dict):
            st.markdown(f"---")
            st.markdown(f"***Entry {exp_idx + 1}***")
            company = st.text_input("Company", exp_item.get('company', ""), key=f"exp_co_{exp_idx}")
            role = st.text_input("Role/Title", exp_item.get('role', ""), key=f"exp_role_{exp_idx}")
            date = st.text_input("Date Range", exp_item.get('date', ""), key=f"exp_date_{exp_idx}")

            bullets = exp_item.get('bullets', [])
            updated_bullets = []
            for b_idx, bullet in enumerate(bullets):
                b_val = st.text_input(f"Bullet {b_idx+1}", bullet, key=f"exp_b_{exp_idx}_{b_idx}")
                if b_val.strip():
                    updated_bullets.append(b_val)

            # Add new bullet
            new_bullet = st.text_input("➕ Add Bullet", "", key=f"exp_newb_{exp_idx}", placeholder="Type new bullet point...")
            if new_bullet.strip():
                updated_bullets.append(new_bullet.strip())

            updated_experience.append({
                'company': company, 'role': role, 'date': date,
                'bullets': updated_bullets
            })
        else:
            val = st.text_input(f"Experience {exp_idx+1}", str(exp_item), key=f"exp_legacy_{exp_idx}")
            if val.strip():
                updated_experience.append(val)

    st.markdown("---")
    st.markdown(f"***➕ Add New Experience Block***")
    new_exp_co = st.text_input("Company", "", key="exp_new_co", placeholder="e.g. Acme Corp")
    new_exp_role = st.text_input("Role", "", key="exp_new_role", placeholder="e.g. Software Engineer")
    new_exp_date = st.text_input("Date", "", key="exp_new_date", placeholder="e.g. Jan 2020 - Present")
    if new_exp_co.strip() or new_exp_role.strip():
        updated_experience.append({
            'company': new_exp_co.strip(), 'role': new_exp_role.strip(), 'date': new_exp_date.strip(),
            'bullets': []
        })

    data_s['experience'] = updated_experience

# --- Education ---
with st.expander("📄 Education", expanded=True):
    heading_edu = data_s.get('original_headings', {}).get('education', 'Education')
    new_h_edu = st.text_input("Section Heading", heading_edu, key="ed_h_edu")
    data_s['original_headings']['education'] = new_h_edu

    education = data_s.get('education', [])

```

```

updated_education = []

for edu_idx, edu_item in enumerate(education):
    if isinstance(edu_item, dict):
        st.markdown(f"---")
        st.markdown(f"***Entry {edu_idx + 1}***")
        school = st.text_input("School/University", edu_item.get('school', ''), key=f'edu_sch_{edu_idx}')
        degree = st.text_input("Degree", edu_item.get('degree', ''), key=f'edu_deg_{edu_idx}')
        edu_date = st.text_input("Date", edu_item.get('date', ''), key=f'edu_date_{edu_idx}')
        updated_education.append({'school': school, 'degree': degree, 'date': edu_date})
    else:
        val = st.text_input(f"Education {edu_idx+1}", str(edu_item), key=f'edu_legacy_{edu_idx}')
        if val.strip():
            updated_education.append(val)

st.markdown("---")
st.markdown("***+ Add New Education Block***")
new_edu_sch = st.text_input("School", "", key="edu_new_sch", placeholder="e.g. MIT")
new_edu_deg = st.text_input("Degree", "", key="edu_new_deg", placeholder="e.g. BS Computer Science")
new_edu_date = st.text_input("Date", "", key="edu_new_date", placeholder="e.g. Graduated 2024")
if new_edu_sch.strip() or new_edu_deg.strip():
    updated_education.append({
        'school': new_edu_sch.strip(), 'degree': new_edu_deg.strip(), 'date': new_edu_date.strip()
    })

data_s['education'] = updated_education

# --- Skills ---
with st.expander("👤 Skills", expanded=True):
    heading_skl = data_s.get('original_headings', {}).get('skills', 'Technical Skills')
    new_h_skl = st.text_input("Section Heading", heading_skl, key="ed_h_skl")
    data_s['original_headings']['skills'] = new_h_skl
    skills_csv = st.text_area("Skills (comma-separated)", ", ".join(data_s.get('skills', [])), key="ed_skills")
    data_s['skills'] = [s.strip() for s in skills_csv.split(",") if s.strip()]

# --- Certifications ---
with st.expander("🏆 Certifications", expanded=False):
    heading_cert = data_s.get('original_headings', {}).get('certifications', 'Certifications')
    new_h_cert = st.text_input("Section Heading", heading_cert, key="ed_h_cert")
    data_s['original_headings']['certifications'] = new_h_cert

    certs = data_s.get('certifications', [])
    updated_certs = []
    for c_idx, cert in enumerate(certs):
        c_val = st.text_input(f"Certification {c_idx+1}", str(cert), key=f'cert_{c_idx}')
        if c_val.strip():
            updated_certs.append(c_val.strip())
    new_cert = st.text_input("👤 Add Certification", "", key="cert_new", placeholder="e.g. AWS Solutions Architect – 2025")
    if new_cert.strip():
        updated_certs.append(new_cert.strip())
    data_s['certifications'] = updated_certs

# --- Projects ---
with st.expander("🚀 Projects", expanded=False):
    heading_proj = data_s.get('original_headings', {}).get('projects', 'Projects')
    new_h_proj = st.text_input("Section Heading", heading_proj, key="ed_h_proj")
    data_s['original_headings']['projects'] = new_h_proj

    projects = data_s.get('projects', [])
    updated_projects = []
    for p_idx, proj in enumerate(projects):
        if isinstance(proj, dict):
            st.markdown(f"---")
            st.markdown(f"***Project {p_idx + 1}***")
            p_name = st.text_input("Project Name", proj.get('name', ''), key=f'proj_name_{p_idx}')
            p_desc = st.text_area("Description", proj.get('description', ''), key=f'proj_desc_{p_idx}', height=80)
            p_tech = st.text_input("Technologies", proj.get('tech', ''), key=f'proj_tech_{p_idx}')
            updated_projects.append({'name': p_name, 'description': p_desc, 'tech': p_tech})
        else:
            p_val = st.text_input(f"Project {p_idx+1}", str(proj), key=f'proj_legacy_{p_idx}')

```

```

        if p_val.strip():
            updated_projects.append(p_val.strip())

    st.markdown("---")
    st.markdown("*** 📌 Add New Project ***")
    new_p_name = st.text_input("Project Name", "", key="proj_new_name", placeholder="My Awesome Project")
    new_p_desc = st.text_area("Description", "", key="proj_new_desc", height=60, placeholder="Brief description...")
    new_p_tech = st.text_input("Technologies", "", key="proj_new_tech", placeholder="Python, React, AWS")
    if new_p_name.strip():
        updated_projects.append({'name': new_p_name.strip(), 'description': new_p_desc.strip(), 'tech': new_p_tech.strip()})
    data_s['projects'] = updated_projects

# --- Save All Changes ---
st.markdown("<br>", unsafe_allow_html=True)
if st.button("💾 Save All Changes & Update Preview", type="primary", use_container_width=True):
    # Final merge of updated data
    st.session_state.processed_data[sel_s] = data_s
    st.success(f"Successfully archived changes for {sel_s}!")
    st.toast("Neural preview updated with new data.")
    st.rerun()

# --- Download Built PDF ---
st.markdown("<br>", unsafe_allow_html=True)
with st.container(border=True):
    st.markdown("##### 📄 Export Asset")
    st.caption("Generate a hardened PDF from the current Live Preview layout.")

    # Convert the live HTML into a PDF byte stream
    pdf_html = generate_html_resume(data_s, st.session_state.selected_template, for_pdf=True)
    pdf_bytes = convert_html_to_pdf(pdf_html)

    if pdf_bytes:
        st.download_button(
            label="📄 Download Premium PDF",
            data=pdf_bytes,
            file_name=f"{data_s.get('name', 'Candidate').replace(' ', '_')}_{st.session_state.selected_template.replace(' ', '')}.pdf",
            mime="application/pdf",
            use_container_width=True,
            type="secondary"
        )
    else:
        st.error("Engine failed to assemble the PDF binary.")

# =====
# TAB 5: LaTeX HUB
# =====
with main_tabs[4]:
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:
        sel_l = st.selectbox("LaTeX Project:", list(st.session_state.processed_data.keys()), key="lx_v3")
        data_l = st.session_state.processed_data[sel_l]
        col_l, col_r = st.columns(2)
        with col_l:
            code = st.text_area("Source (.tex)", value=generate_latex_resume(data_l), height=600)
            if st.button("🔥 COMPILE"):
                pdf = compile_latex_to_pdf(code)
                if pdf: st.session_state[f"pdf_{sel_l}"] = pdf; st.success("Ready!")
        with col_r:
            if f"pdf_{sel_l}" in st.session_state:
                st.markdown(f"<embed src='data:application/pdf;base64,{base64.b64encode(st.session_state[f'pdf_{sel_l}']).decode()}' width='100%' height='600' type='application/pdf'>", unsafe_allow_html=True)
            st.download_button("Download", st.session_state[f"pdf_{sel_l}"], f"{sel_l}.pdf")

# =====
# TAB 6: AI REVIEW
# =====
with main_tabs[5]:
    st.header("🔍 Audit & Quality Review")
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:

```

```

sel_r = st.selectbox("Select Asset for Deep Audit:", list(st.session_state.processed_data.keys()), key="rev_v3")
data_r = st.session_state.processed_data[sel_r]

review_tab1, review_tab2 = st.tabs(["🚩 Bias Audit", "💡 Performance Grading"])

with review_tab1:
    st.write("### AI-Powered Bias Suggestions")
    bias_flags = cached_detect_bias(data_r.get('full_text', ""))
    if bias_flags:
        for flag in bias_flags:
            with st.container(border=True):
                st.markdown(f"***Potential Issue:** {flag['type']}")
                st.markdown(f"***Detected Evidence:** {flag['match']}")
                st.info(f"💡 **Suggestion:** {flag['suggestion']}")
    else:
        st.success("No critical bias triggers detected in the current scan.")

with review_tab2:
    st.write("### 💡 STAR-Method Performance Analysis")
    graded_bullets = grade_bullet_points(data_r.get('experience', []))
    for g in graded_bullets:
        with st.expander(f"{g['star_label']} — {g['line'][:60]}...", expanded=False):
            st.write(f"***Bullet:** {g['line']}")
            st.write(f"***Action Verb Strength:** {g['verb_strength']}")
            if g['suggestion']: st.warning(f"💡 {g['suggestion']}")
            st.write(f"***Quantifiable Metrics:** {'✅ Yes' if g['has_metric'] else '❌ No'}")
            st.write("***STAR Score Components:**")
            st.write(", ".join(g['star_reasons']))

# =====
# TAB 7: JOB INTELLIGENCE
# =====

with main_tabs[6]:
    st.header("🏢 Intelligent Job Market")
    if not st.session_state.processed_data: st.info("Run scan first.")
    else:
        sel_j = st.selectbox("Select Asset for Market Mapping:", list(st.session_state.processed_data.keys()), key="ji_v3")
        data_j = st.session_state.processed_data[sel_j]
        st.write("### Recommended Roles:", ", ".join(suggest_roles(data_j.get('skills', []))))

        jobs = cached_fetch_live_jobs_b64(data_j.get('skills', []))
        j_cols = st.columns(2)
        for idx, j in enumerate(jobs[:8]):
            with j_cols[idx % 2]:
                # Modern card for job
                st.markdown(f"""
                <div class="responsive-flex-card" style="background:{accent}08; border:1px solid {accent}1a; border-radius:15px; padding:20px; margin-
                bottom:15px; display:flex; align-items:center; gap:15px;">
                <div style="background:white; border-radius:10px; padding:10px; box-shadow:0 4px 6px rgba(0,0,0,0.05); width:64px; height:64px;
                display:flex; justify-content:center; align-items:center;">
                
                </div>
                <div style="flex-grow:1;">
                <h4 style="margin:0; font-size:1.1rem; color:{accent};">{j['title']}</h4>
                <p style="margin:5px 0; font-weight:600; opacity:0.9; font-size:0.95rem;">🏢 {j['company']}</p>
                <p style="margin:0; opacity:0.7; font-size:0.85rem;">📍 {j['location']} &nbsp; • &nbsp;🕒 {j.get('posted', 'Recently')}</p>
                <div style="margin-top:8px;"><span style="background:{accent}22; padding:3px 10px; border-radius:20px; font-size:0.8rem; font-
                weight:700;">{j['salary']}</span></div>
                </div>
                </div>
                """, unsafe_allow_html=True)
            st.link_button(f"Apply via {j['platform']}", j['url'], use_container_width=True)
            st.write("") # Spacer

# =====
# TAB 8: MOCK INTERVIEW (Voice + AI)
# =====

with main_tabs[7]:

```

```

st.markdown(f"<div style='background: {accent}11; padding:30px; border-radius:20px; border:1px solid {accent}33; margin-bottom:20px;'><h2
style='margin:0;'><img alt='Neural Interview Session'></h2></div>", unsafe_allow_html=True)

i_col, c_col = st.columns([1, 2], gap="large")
with i_col:
    if os.path.exists("assets/natalia.png"):
        st.image("assets/natalia.png", caption="Virtual HR Specialist: Natalia", use_container_width=True)
    else:
        st.markdown("<div style='text-align:center; font-size:5rem;'><img alt='Natalia'></div>", unsafe_allow_html=True)
        st.caption("Virtual HR Specialist: Natalia")

input_mode = st.radio("Input Mode", ["🗨️ Text", "🎤 Voice"], horizontal=True)
if st.button("Start New Session", type="primary", use_container_width=True):
    st.session_state.chat = [{"role": "assistant", "content": "Hello! I'm Natalia from HR. I've reviewed your profile and I'm impressed. Let's start — can you briefly introduce yourself and tell me what excites you about this role?"}]
    st.rerun()

with c_col:
    if "chat" not in st.session_state: st.session_state.chat = []

    chat_box = st.container(height=500)
    with chat_box:
        for m in st.session_state.chat:
            with st.chat_message(m['role']):
                st.write(m['content'])
            if m['role'] == "assistant":
                if m == st.session_state.chat[-1]:
                    try:
                        tts = gTTS(text=m['content'], lang='en')
                        with tempfile.NamedTemporaryFile(delete=False, suffix=".mp3") as fp:
                            tts.save(fp.name)
                            st.audio(fp.name, format="audio/mp3")
                    except Exception:
                        pass

    if input_mode == "🗨️ Text":
        p = st.chat_input("State your response...")
        if p:
            st.session_state.chat.append({"role": "user", "content": p})
            with st.spinner("Natalia is thinking..."):
                response = generate_natalia_response(p, st.session_state.chat)
            st.session_state.chat.append({"role": "assistant", "content": response})
            st.rerun()
    else:
        # Voice mode using Streamlit audio_input + SpeechRecognition
        st.markdown("##### 🎤 Voice Input — Record your answer")
        st.caption("Click the microphone below, speak your answer. Natalia will listen and respond.")

        audio_data = st.audio_input("🎧 Record your answer", key="voice_recorder")

        if audio_data is not None:
            # Check if this audio instance has already been processed (prevent loop)
            audio_bytes = audio_data.getvalue()
            audio_hash = hash(audio_bytes)

            if "last_processed_audio" not in st.session_state or st.session_state.last_processed_audio != audio_hash:
                with st.spinner("🎧 Transcribing neural signals..."):
                    try:
                        r = sr.Recognizer()
                        with sr.AudioFile(audio_data) as source:
                            audio = r.record(source)
                            text = r.recognize_google(audio)

                        if text:
                            st.session_state.chat.append({"role": "user", "content": text})
                            with st.spinner("Natalia is reflecting on your answer..."):
                                response = generate_natalia_response(text, st.session_state.chat)
                            st.session_state.chat.append({"role": "assistant", "content": response})
                            st.session_state.last_processed_audio = audio_hash
                            st.rerun()

```

```

except sr.UnknownValueError:
    st.error("Sorry, I couldn't understand the audio. Please try speaking clearer.")
except sr.RequestError:
    st.error("Neural transcription service is currently offline.")
except Exception as e:
    st.error(f"Error: {str(e)}")

if audio_data is not None:
    st.audio(audio_data, format="audio/wav")

if st.button("🗣️ Transcribe & Send to Natalia", type="primary", use_container_width=True):
    with st.spinner("🔄 Transcribing your speech..."):
        try:
            recognizer = sr.Recognizer()
            # Convert Streamlit audio BytesIO to AudioData
            audio_bytes = audio_data.getvalue()
            audio_file = io.BytesIO(audio_bytes)
            with sr.AudioFile(audio_file) as source:
                audio_record = recognizer.record(source)
            transcript = recognizer.recognize_google(audio_record)

            if transcript.strip():
                st.success(f"***Transcribed:** {transcript}")
                st.session_state.chat.append({"role": "user", "content": f"🗣️ {transcript}"})
                response = generate_natalia_response(transcript, st.session_state.chat)
                st.session_state.chat.append({"role": "assistant", "content": response})
                st.rerun()
            else:
                st.warning("Could not understand the audio. Please try again.")
        except sr.UnknownValueError:
            st.warning("🔊 Could not understand the audio. Please speak clearly and try again.")
        except sr.RequestError as e:
            st.error(f"Speech recognition service error: {e}")
        except Exception as e:
            st.error(f"Error processing audio: {str(e)}")
    else:
        st.info("👉 Click the microphone button above to start recording your answer.")

st.divider()
st.caption("💡 **Tip:** You can also type your response below as a fallback.")
voice_text = st.text_input("🗣️ Type your response:", key="voice_manual_input", placeholder="Type here if microphone isn't working...")
if st.button("💬 Send Text Response", use_container_width=True):
    text = voice_text.strip()
    if text:
        st.session_state.chat.append({"role": "user", "content": f"🗣️ {text}"})
        response = generate_natalia_response(text, st.session_state.chat)
        st.session_state.chat.append({"role": "assistant", "content": response})
        st.rerun()
    else:
        st.warning("Please type or record a response first.")

# =====
# TAB 9: FEEDBACK (Auto-populated from signed-in user)
# =====
with main_tabs[8]:
    st.header("🔄 Intel Feedback Loop")
    with st.container(border=True):
        # Auto-populate user info from profile
        profile = st.session_state.user_profile or {}

        st.markdown(f"""
        <div style='background: {accent}11; padding:15px; border-radius:12px; border:1px solid {accent}33; margin-bottom:15px;'>
        <p style='margin:0;'><strong>👤 Submitting as:</strong> {profile.get('name', 'Guest')} • {profile.get('email', 'Not signed in')} • {profile.get('phone',
        'N/A')}</p>
        </div>
        """, unsafe_allow_html=True)

    f_text = st.text_area("Provide your feedback or report an anomaly...", height=150)

```

```

feedback_type = st.selectbox("Feedback Category", ["General", "Bug Report", "Feature Request", "UI/UX", "Performance"])
rating = st.slider("Overall Satisfaction", 1, 5, 4, help="1 = Poor, 5 = Excellent")

if st.button("Transmit Feedback", type="primary"):
    if not f_text.strip():
        st.warning("Please enter some feedback before submitting.")
    else:
        feedback_data = {
            "user_id": profile.get("email", "Anonymous"),
            "name": profile.get("name", "Guest"),
            "email": profile.get("email", "Not Provided"),
            "phone": profile.get("phone", "N/A"),
            "timestamp": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
            "category": feedback_type,
            "rating": int(rating),
            "feedback": str(f_text)
        }
        # Save to file
        os.makedirs("data", exist_ok=True)
        with open("data/user_feedback_records.txt", "a") as f:
            f.write(f'--- FEEDBACK [{feedback_data["timestamp"]} ---\n')
            f.write(f'USER: {feedback_data["name"]} ( {feedback_data["email"]} ) | PHONE: {feedback_data["phone"]}\n')
            f.write(f'CATEGORY: {feedback_data["category"]} | RATING: {feedback_data["rating"]}/5\n')
            f.write(f'MESSAGE: {feedback_data["feedback"]}\n')
            f.write("-" * 50 + "\n\n")

        st.success("✅ Feedback stored in neural archives. Thank you!")
        st.balloons()

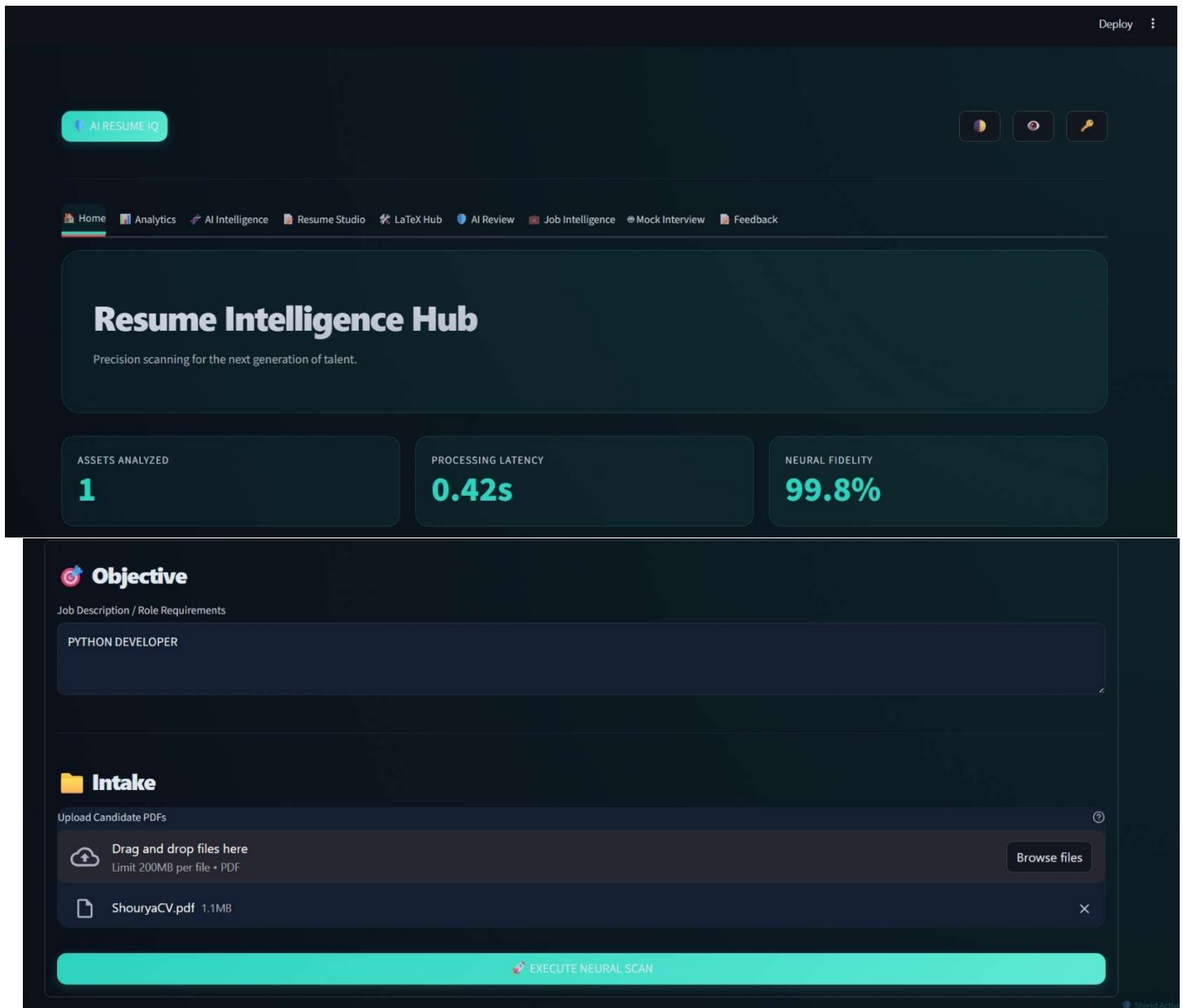
def main():
    apply_adaptive_theme()
    accent = "#2DD4BF"

    render_top_nav(accent)
    if st.session_state.current_page == "admin":
        if not st.session_state.is_signed_in:
            render_auth_page(accent)
        else:
            render_admin_page(accent)
    elif st.session_state.current_page == "user_dashboard":
        if not st.session_state.is_signed_in:
            render_auth_page(accent)
        else:
            render_user_dashboard(accent)
    else:
        render_home_page(accent)

if __name__ == "__main__": main()

```

8.2 Output



Chapter 9

Conclusion

The development of the AI Resume Intelligence Hub represents a significant step toward modernizing and improving the traditional recruitment process. In today's fast-paced digital environment, organizations face the challenge of handling a large volume of job applications while ensuring that the most suitable candidates are selected efficiently and fairly. This system addresses these challenges by integrating advanced technologies such as Artificial Intelligence, Natural Language Processing, and semantic analysis into a unified platform. The result is a comprehensive solution that enhances both the efficiency and accuracy of recruitment while also improving the experience for candidates.

The system successfully automates the process of resume screening, which is traditionally time-consuming and prone to human bias. By converting unstructured resume data into structured information, the platform enables a more objective evaluation of candidates. The use of NLP techniques ensures that important details such as skills, education, and experience are accurately extracted and analyzed. Furthermore, the implementation of semantic matching allows the system to understand the context and meaning of text rather than relying solely on keyword matching. This leads to more accurate candidate-job alignment and reduces the chances of overlooking qualified applicants.

Another important achievement of the system is the implementation of a robust candidate ranking mechanism. By assigning similarity scores based on semantic analysis, the system provides recruiters with a prioritized list of candidates. This not only saves time but also improves decision-making by highlighting the most relevant profiles. The ranking system ensures that recruitment becomes more data-driven and less dependent on subjective judgment.

In addition to recruitment functionalities, the system also focuses on candidate development. Features such as resume generation and mock interviews provide valuable support to job seekers. The resume generation module helps candidates create professional, well-structured, and ATS-friendly resumes, increasing their chances of being shortlisted. The mock interview module, powered by speech and AI technologies, offers an interactive platform for candidates to practice and improve their communication skills. This holistic approach makes the system beneficial for both recruiters and candidates, creating a balanced and efficient ecosystem.

The modular architecture of the system is another key strength. Each component, such as resume parsing, semantic matching, ranking, and interview simulation, operates independently while maintaining seamless integration. This design ensures flexibility and scalability, allowing the system to handle increasing workloads and adapt to future requirements. The use of containerization technologies further enhances the system's ability to be deployed across different environments without compatibility issues.

Despite its advantages, the system also has certain limitations. Handling highly unstructured or poorly formatted resumes can be challenging, as variations in format may affect the accuracy of text extraction and analysis. Additionally, the performance of AI models depends on the quality and diversity of the data used for training. Bias in training data can lead to biased outcomes, which must be carefully addressed through ethical AI practices. Computational complexity is another consideration, as advanced models require significant processing power and resources.

However, these limitations also present opportunities for improvement and innovation. Continuous advancements in AI and NLP technologies provide a strong foundation for enhancing the system's capabilities.

By incorporating more advanced models and expanding training datasets, the accuracy and reliability of the system can be further improved.

9.1 Future Scope

The future scope of the AI Resume Intelligence Hub is extensive, with numerous opportunities for enhancement and expansion. One of the primary areas of development is the integration of more advanced AI models. As technology evolves, newer models with improved language understanding and reasoning capabilities can be incorporated into the system. This will enable even more accurate analysis of resumes and job descriptions, as well as more sophisticated feedback generation.

Another important area of future development is the incorporation of multi-language support. Currently, many recruitment systems are limited to specific languages, which restricts their usability in global markets. By integrating multilingual NLP models, the system can be expanded to support resumes and job descriptions in multiple languages. This will make the platform more inclusive and accessible to a wider audience.

The system can also be enhanced by integrating with external job portals and professional networking platforms. This would allow automatic retrieval of job descriptions and candidate profiles, streamlining the recruitment process further. Integration with platforms such as LinkedIn could provide additional data points for evaluating candidates, such as endorsements, projects, and professional connections.

Another promising area is the development of advanced analytics and visualization tools. By incorporating dashboards and graphical representations, the system can provide deeper insights into recruitment trends, candidate performance, and skill gaps. These insights can help organizations make strategic decisions and improve their hiring processes. Predictive analytics can also be introduced to forecast candidate success and retention based on historical data.

The mock interview module can be further enhanced by incorporating video analysis and emotion detection. This would allow the system to evaluate not only the content of responses but also non-verbal cues such as facial expressions and body language. Such features would provide a more comprehensive assessment of candidate performance and help users prepare more effectively for real interviews.

Another potential improvement is the inclusion of personalized learning and career guidance features. Based on the analysis of resumes and interview performance, the system can recommend courses, certifications, and skill development opportunities. This would transform the platform into a complete career development tool, supporting users beyond the recruitment process.

Scalability and cloud integration also present significant opportunities for future development. By deploying the system on cloud platforms, it can handle larger datasets and support a growing number of users. Cloud-based solutions also enable real-time processing and collaboration, making the system more efficient and accessible. Security and data privacy will continue to be important considerations in future development. Implementing advanced encryption techniques, secure authentication mechanisms, and compliance with data protection regulations will ensure that user data remains safe and confidential.

Finally, the system can be extended to support additional recruitment functionalities, such as automated job description generation, candidate recommendation systems, and AI-driven hiring decisions. These enhancements will further streamline the recruitment process and make it more intelligent and efficient.

Chapter 10

References

- [1] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 2nd ed., Pearson, 2009.
- [2] S. Bird, E. Klein and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.
- [3] M. Honnibal and I. Montani, “spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing”, 2017.
- [4] S. Bird, “NLTK: The Natural Language Toolkit”, *Proceedings of the ACL Interactive Poster and Demonstration Sessions*, 2006.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- [6] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient Estimation of Word Representations in Vector Space”, *International Conference on Learning Representations (ICLR)*, 2013.
- [7] A. Müller and S. Guido, *Introduction to Machine Learning with Python*, O'Reilly Media, 2016.
- [8] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library”, *Advances in Neural Information Processing Systems*, 2019.
- [9] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System”, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [10] Streamlit Inc., “Streamlit Documentation”, 2023.
- [11] Docker Inc., “Docker: Enterprise Container Platform”, 2023.
- [12] L. Lamport, *LaTeX: A Document Preparation System*, Addison-Wesley, 1994.

- [13] J. Brownlee, *Machine Learning Mastery with Python*, Machine Learning Mastery, 2016.
- [14] Google, “Google Text-to-Speech (gTTS) API Documentation”, 2023.
- [15] Uberi, “SpeechRecognition Library Documentation”, 2023.